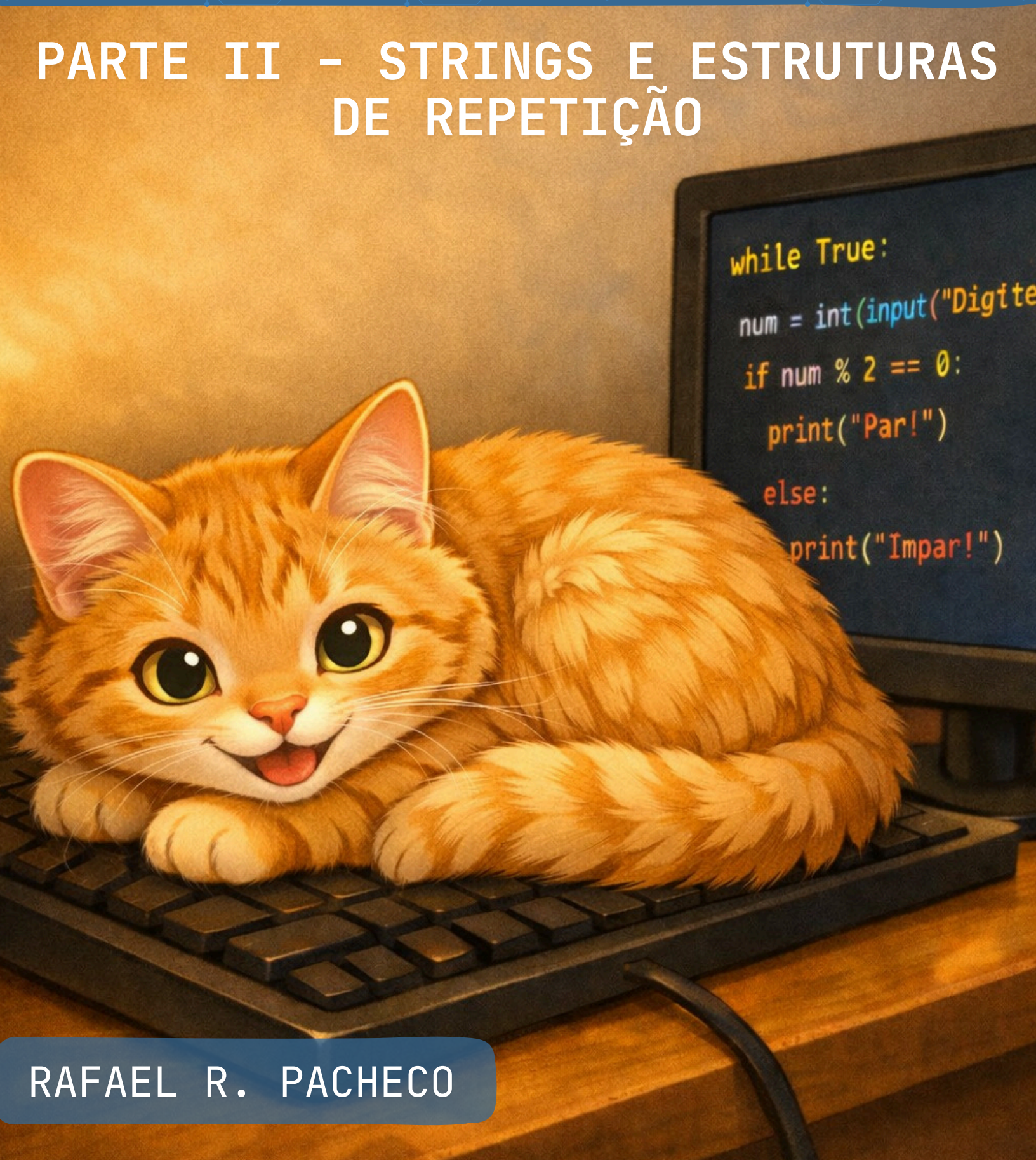
Tutorial Didático de Python

PARTE II - STRINGS E ESTRUTURAS DE REPETIÇÃO



RAFAEL R. PACHECO

SUMÁRIO

Introdução.....1

CAPÍTULO 1 – Strings.....2

1.1 Revisando os Conceitos.....2

1.2 Características e Propriedades.....2

1.2.1 Estrutura de Dados.....3

1.2.2 Sequência Ordenada.....4

1.2.3 Imutabilidade.....4

1.2.4 Iterável.....6

1.3 Índices da String.....6

1.3.1 Acessando Caracteres Individuais.....7

1.3.2 Acessando Partes – Slicing.....11

1.4 Métodos de String.....18

1.4.1 Capitalização – upper(), lower(), capitalize(), title() e swapcase().....18

1.4.2 Análise de Conteúdo – isalpha(), isdigit(), isalnum(), isspace().....21

1.4.3 Busca e Contagem – find(), count(), startswith(), endswith().....22

1.4.4 Modificação da String.....23

1.4.5 Métodos join() e split().....25

1.5 Função len().....26

CAPÍTULO 2 – Exercícios com Strings.....28

2.1 Introdução.....28

2.2 Slicing – Frutas em Ordem Alfabética.....28

2.3 Analisando a String.....30

2.4 Detectando Palíndromos.....32

CAPÍTULO 3 – Estruturas de Repetição: Laço For.....37

3.1 Introdução aos Laços de Repetição.....37

3.2 Laço For – For Loop.....37

3.3 Sintaxe do For.....38

3.4 Função range().....41

3.4.1 Usando range() Para Contar Repetições.....42

3.4.2 Usando range() Para Exibir Números.....43

3.4.3 Nome das Variáveis de Controle no range() – Convenções.....45

3.4.4 A range() Aceita Variáveis Numéricas.....47

3.5 Laço For Aninhado – Nested For Loop.....49

3.6 Manipulando os Laços For – break e continue.....54

3.6.1 Comando break.....54

3.6.2 Comando continue.....56

3.7 For e Else.....58

CAPÍTULO 4 – Exercícios com for Loop.....60

4.1 Introdução.....60

4.2 Somando Números Digitados.....60

4.3 Maior e Menor Número Digitado.....62

CAPÍTULO 5 – Estruturas de Repetição: Laço While.....65

5.1 Definição e Sintaxe.....65

5.2 Diferenças Entre For e While e Quando Usar.....67

5.3 Break e Continue.....69

5.4 While True – Loop Infinito.....69

5.4.1 Problema do Loop Infinito e Como Evitar.....71

5.4.2 Quando Usar Loop Infinito.....71

5.5 Laço While Aninhado – Nested While Loop…….75

5.6 While e Else.....78

CAPÍTULO 6 – Exercícios com While Loop.....79

6.1 Validação de Entrada – Básico.....79

6.2 Validação de Entrada – Aprimorando.....80

6.3 Caixa Eletrônico.....82

6.4 Considerações Finais.....84

Introdução

Este tutorial foi desenvolvido como projeto de extensão no curso de Ciência da Computação da UNIP, sendo, portanto, totalmente gratuito. O objetivo desse tutorial é ensinar a linguagem de programação Python de forma simples e didática, permitindo que qualquer pessoa possa aprender a programar.

Assim como a primeira parte, também temos por objetivo manter uma linguagem simples e acessível, com foco na didática e nos exemplos para que o material sirva como apoio aos estudos universitários e outros cursos de Python.

Esta segunda parte dá continuidade aos conceitos básicos de Python, focando em strings e laços de repetição (for e while).

Bons estudos!

CAPÍTULO 1 – Strings

1.1 Revisando Os Conceitos

Conforme estudamos na parte I do tutorial, strings consistem em um dos tipos básicos do Python, representando texto ou sequência de caracteres, ainda que seja uma string vazia.

Internamente, o Python reconhece esse tipo como str.

Strings são extremamente importantes e fazem parte do dia a dia do programador. Vamos aprofundar os conhecimentos que temos sobre esse tipo básico e entendermos o que podemos fazer com ele.

1.2 Características e Propriedades

As strings fazem parte do que chamamos “estrutura de dados”. Especificamente é uma estrutura de dados sequencial e ordenada, pois um texto nada mais é que uma coleção de caracteres guardados em sequência e em ordem em uma variável.

Strings são imutáveis, não sendo possível alterá-las após a sua declaração. Também são sujeitas a iteração e podemos acessar cada caractere individualmente ou partes do texto em questão.

Vamos explicar tudo de forma simples e didática!

1.2.1 Estrutura de Dados

Chamamos de estruturas de dados as formas utilizadas para armazenar, organizar e manipular informações. O Python possui diversos tipos de estruturas que desempenham essa função, como as listas, tuplas, dicionários e **strings**.

O estudo aprofundado de estrutura de dados será feito na parte 3 desse tutorial, sendo apenas necessário saber que a string é uma estrutura de dados, além de ser um tipo básico.

Devido a essa característica, podemos pegar qualquer string e acessar os seus caracteres individualmente, ou pegar parte deles. Vamos pensar na seguinte string:

```
1    exemplo = "Hello, World!"
2
```

Observe que a variável exemplo é do tipo string e armazena diversas informações – um grupo de caracteres que juntos formam um texto que podemos compreender.

Conforme foi explicado, podemos acessar cada caractere individualmente. Podemos pegar somente o “H”, ou então a vírgula “,” ou até mesmo o espaço entre as palavras “ “. Também é possível acessar pedaços da string, como “Hello” ou “Wor” ou “, World!”

Não se preocupe! Nesse capítulo iremos aprender como fazer isso! O importante agora é entender que fato de ser uma estrutura de dados é que permite a string ter esse comportamento.

1.2.2 Sequência Ordenada

Quando escrevemos uma string, a ordem dos caracteres importa. Eles sempre estarão em ordem. É exatamente como escrever uma palavra ou um texto. A ordem dos caracteres vai determinar o que foi escrito.

Então, a string “AE” não é o mesmo que a string “EA”, pois, ainda que ambas possuam exatamente os mesmos caracteres, eles estão em ordem diferente e isso é uma diferença fundamental.

Esse conceito também será importante para quando quisermos acessar esses caracteres ou partes da string, conforme mencionamos no item anterior.

1.2.3 Imutabilidade

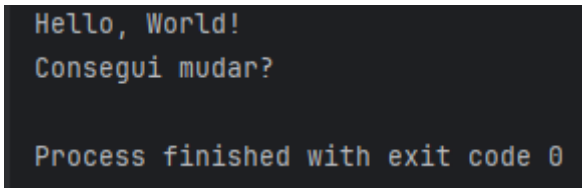
Uma vez declarada e armazenada em uma variável, a string não pode ser mudada. O seu conteúdo não pode ser alterado, sendo impossível adicionar, remover ou substituir novos caracteres.

Se precisarmos “alterar” algo, temos de criar uma nova string e atribuí-la a uma nova variável. Podemos, também, realizar a atribuição novamente na mesma variável, o que irá substituir (mas não mudar) o conteúdo da string velha pela nova.

Vamos explicar isso com um código muito simples:

```
1  exemplo = "Hello, World!"
2  print(exemplo)
3
4  exemplo = "Consegui mudar?"
5  print(exemplo)
6
```

Resultado dos prints no console:



O processo que seguimos foi o seguinte: primeiramente criamos a variável “exemplo” com o texto do olá, mundo como conteúdo. Quando exibimos o print, o “Hello, World!” foi exibido no console.

Posteriormente, usamos a mesma variável exemplo, mas agora atribuímos o valor “Consegui mudar?” a ele e logo em seguida mandamos printar no console.

Aparentemente conseguimos mudar o conteúdo da string, não? Provamos que a imutabilidade está errada?

NÃO! Nós não mudamos string! Apesar de ser algo que pareça sutil, o que fizemos foi inicializar novamente a variável “exemplo”. Quando criamos a variável com o conteúdo “Hello, World!” ela foi armazenada na memória. O comando print fez o Python exibir o que estava armazenado na memória no console.

Agora quando tentamos mudar o conteúdo da variável “exemplo” na linha 4 o que realmente aconteceu foi o seguinte: o Python APAGOU a primeira atribuição que fizemos (linha 1) e a variável “exemplo” deixou de existir.

Em seguida, foi inicializada nova variável (com o mesmo nome por coincidência) e salva na memória. Quando houve o segundo print (linha 5), não houve mudança de conteúdo da variável (a string não mudou!). O que aconteceu é que foi feita uma NOVA atribuição. É OUTRA string completamente diferente da primeira.

Nesse exemplo nosso, a primeira variável “exemplo” da linha 1 foi completamente destruída e perdida. Não temos mais como acessá-la depois da nova atribuição realizada na linha 4.

Esse exemplo pode parecer um pouco confuso no começo, mas é essencial entender isso para saber que strings são imutáveis. Quando quisermos “mudar” ou “alterar” uma string, precisamos salvá-la em outra variável, realizando nova atribuição.

1.2.4 Iterável

As strings são iteráveis. Essa propriedade permite que percorramos caractere a caractere da string usando um laço de repetição (como o laço for).

Estudaremos esse laço no próximo capítulo e mostraremos exemplos de strings sendo iteradas. Não se preocupe com isso no momento, apenas saiba que strings são iteráveis e que isso fará sentido no próximo capítulo!

1.3 Índices da String

Como as strings são estruturas de dados sequenciais e ordenadas, o Python organiza os seus caracteres em índices. Todo e qualquer caractere da string, inclusive espaços vazios, são indexados.

E o que significa esse tipo de organização? Ser indexado?

Isso significa que o Python conta quantos caracteres compõem a string. Ele sabe o tamanho da string e atribui uma posição numérica para cada caractere, seguindo uma sequência que começa do 0 e vai subindo de 1 em 1.

No nosso exemplo da string “Hello, World!”, o índice do “H” seria 0, o do “e” seria 1, a vírgula seria 5 e o espaço seria 6. O ponto de exclamação, último caractere da nossa string, seria 12. Basta contar os caracteres começando do 0.

Esse tipo de organização é extremamente útil, pois permite ao programador acessar os caracteres individualmente, ou acessar conjuntos de caracteres (pedaços da string), de forma simples e facilitada, pois basta saber o índice desejado (ou, no caso de parte, o range, o intervalo, desses índices).

ATENÇÃO: É muito importante lembrar que a contagem sempre começa do 0! O primeiro caractere de qualquer string sempre irá ocupar o índice 0!

1.3.1 Acessando Caracteres Individuais

Agora que sabemos que é possível acessar os caracteres da string, vamos aprender como fazemos para acessar um caractere qualquer de forma individual.

É muito simples. Basta usar a seguinte fórmula: variável[índice].

Variável é o nome da variável que contém a string e dentro das chaves [] temos o número que indica a posição numérica do caractere que queremos acessar. Vamos testar isso no código printando os caracteres usando a fórmula indicada.

```
1    exemplo = "Hello, World!"
2    print(exemplo[0])
3    print(exemplo[1])
4    print(exemplo[5])
5    print(exemplo[6])
6
```

Console:

```
H
e
,


Process finished with exit code 0
```

Exibimos todos os índices indicados, inclusive o espaço vazio que veio depois da vírgula (índice 6).

Podemos usar esse método para salvar os caracteres em outra variável, sem perder a string original. Exemplo:

```
1    exemplo = "Hello, World!"
2
3    caractere = exemplo[1]
4    print(caractere)
5
```

Resultado:

```
e


Process finished with exit code 0
```

Assim, a variável que criamos chamada “caractere” é uma string que recebeu o segundo elemento (posição 1, lembrando que a primeira posição é o 0) do conteúdo da string “exemplo”.

O Python também consegue acessar os índices de trás para frente, começando do último e terminando no primeiro. Da mesma forma que conseguimos acessar os caracteres partindo do 0 até a posição do último caractere da string, é possível começar pelo último e chegar até o primeiro, seguindo uma ordem inversa.

Para fazer isso basta começarmos com a posição [-1]. O Python entende que essa posição pertence ao último caractere da string, sendo que o penúltimo será o -2, o antepenúltimo o -3 e assim sucessivamente, até chegar no primeiro.

Vamos mostrar isso na prática:

```
exemplo = "Hello, World!"
print(exemplo[-1])
print(exemplo[-3])
print(exemplo[-6])
print(exemplo[-7])
print(exemplo[-8])
print(exemplo[-13])
```

Prints:

```
!
l
W

,
H

Process finished with exit code 0
```

Essa funcionalidade é bastante útil, pois permite que acessemos os índices de trás para frente sem nem saber o tamanho da string. Quando o programador deseja acessar o último caractere, ele não precisa saber o tamanho do texto, bastando usar a posição [-1].

E o que acontece se tentarmos acessar um índice que não existe? Pensando no nosso exemplo, a frase “Hello, World!” possui exatamente 13 caracteres (e 13 posições, lembrando que a primeira posição é 0 e não 1!). E se tentarmos acessar a posição [13]?

Vamos testar:

```
exemplo = "Hello, World!"
print(exemplo[13])

```

Console:

```
Traceback (most recent call last):
  File "C:\Users\rafae\PycharmProjects\Tutorial\.venv\strings.py", line 2, in <module>
    print(exemplo[13])
          ~~~~~~~~^^^^
IndexError: string index out of range

Process finished with exit code 1
```

Recebemos uma mensagem de erro! Vamos analisar: na linha 2, ou seja, no comando print(exemplo[13]) o Python detectou um problema. IndexError – erro de índice- o índice da string está fora do intervalo!

Com isso a própria linguagem de programação já respondeu à nossa dúvida: é impossível acessar uma posição que não existe!

Mas, espere! Não havíamos contado os caracteres e nos certificado de que a string tinha exatamente 13 caracteres? Não era para a posição 13 corresponder ao último caractere da string, o ponto de exclamação? **NÃO!** Temos 13 posições, mas a primeira começa do 0! Então nossos índices válidos vão de 0 a 12.

Esse mesmo problema aconteceria caso tentássemos acessar a posição [-14]. Os índices de nosso exemplo devem ir de -1 a -13 na lógica inversa. Vejamos:

```
exemplo = "Hello, World!"
print(exemplo[-14])

```

```
Traceback (most recent call last):
  File "C:\Users\rafae\PycharmProjects\Tutorial\.venv\strings.py", line 2, in <module>
    print(exemplo[-14])
          ~~~~~~~~^^^^^
IndexError: string index out of range

Process finished with exit code 1
```

Exatamente o mesmo erro! Então devemos tomar cuidado para não tentar acessar índices inexistentes, pois o programa irá quebrar se fizermos isso.

1.3.2 Acessando Partes – Slicing

O Python também permite que acessemos partes (ou fatias) de uma string. Esse processo é chamado de *slicing* ou, em português, fatiamento. Da mesma forma que era possível pegar os caracteres de forma individual, o programador também consegue pegar partes da string.

Em outras palavras, é possível que ele acesse somente a palavra “Hello” da nossa string “exemplo”, ou o pedaço “llo, “, ou qualquer outro pedaço que ele desejar. Por isso que essa funcionalidade é chamada de *slicing.* Podemos cortar (fatiar) a nossa string da forma que quisermos, respeitando o tamanho dela, pois já aprendemos que não conseguimos acessar índices inexistentes.

E como fazemos isso? É bem simples, mas vamos prestar atenção para não fazermos confusão!

A fórmula do fatiamento é: variável[início:fim:passo].

a) **Início:** esse parâmetro indica justamente onde vai começar o fatiamento. A partir de qual índice é que vamos pegar o pedaço da nossa string. **O início vai fazer parte do pedaço que pegamos da string.**

ATENÇÃO: Caso deixemos esse parâmetro em branco, o Python vai interpretar que desejamos começar desde o primeiro caractere, ou seja, a primeira posição – índice 0.

b) **Fim:** Esse parâmetro indica o final do fatiamento. Com o início e o fim, o Python vai saber onde começa e onde termina o pedaço. O que estiver entre esses índices será cortado no pedaço. **O fim NÃO vai fazer parte do pedaço que pegamos da string. Em uma string de posições 0 a 12, para que o último caractere seja incluso na fatia nós devemos indicar 13 como fim, mesmo que não exista posição 13!**
ATENÇÃO: Caso deixemos esse parâmetro em branco, o Python vai interpretar que desejamos ir até o fim da string, **incluindo o último caractere!** Ou seja, a última posição e ela inclusa.

c) **Passo:** Esse último parâmetro nos permite modificar a forma que o fatiamento será feito. Por exemplo, podemos apenas pegar os índices pares do nosso intervalo definido no início e fim. Com os exemplos vai ficar mais fácil entender a utilidade do passo.
ATENÇÃO: Caso deixemos esse parâmetro em branco, o Python vai interpretar que o passo será de 1 em 1, ou seja, índice a índice sem pular nenhum. Essa é a forma mais comum de realizarmos o fatiamento

Assim como fizemos quando acessamos os caracteres, os pedaços de strings que pegarmos também podem ser salvos em novas variáveis. Não estamos obrigados a printar elas diretamente.

Agora que sabemos como fazer o *slicing*, vamos estudar diversos exemplos para entendermos bem o funcionamento do fatiamento e fixar como os parâmetros se comportam:

Primeiro exemplo: vamos fatiar a string em sua totalidade e de índice em índice (passo 1):

```
exemplo = "Hello, World!"

print(exemplo[0:13:1])

```

```
Hello, World!

Process finished with exit code 0
```

O que fizemos? Mandamos o Python fatiar a string começando da posição 0 (e ela inclusa), ou seja, do seu primeiro caractere até o caractere na posição 13 (**e ele não incluso, ou seja, foi do intervalo de 0 a 12!**) usando passo 1, ou seja, posição a posição sem pular nada.

Resultado: o fatiamento pegou a string inteira. Esse exemplo é a mesma coisa que simplesmente printar a string inteira – print(exemplo).

ATENÇÃO: Perceba que acessar exemplo[13] vai dar erro, conforme provamos no item 1.3.1, pois os índices dessa string vão só de 0 a 12. Entretanto, como o fatiamento é realizado pegando o intervalo dentre o início (incluso) e o fim (não incluso), para pegar a string até o último caractere devemos adicionar +1 à posição do último item.

Então, para conseguirmos fatiar a string incluindo nesse fatiamento o último caractere dela sempre temos de fazer a posição desse último + 1. Se tivéssemos colocado 12 em vez de 13 o ponto de exclamação não faria parte do fatiamento. Veja:

```
exemplo = "Hello, World!"

print(exemplo[0:13:1])
print(exemplo[0:12:1])

```

```
Hello, World!
Hello, World

Process finished with exit code 0
```

Essa é a grande “dificuldade” do fatiamento. Não podemos esquecer que o índice que passamos no parâmetro fim não vai fazer parte do fatiamento.

Outro ponto a ser destacado nesse primeiro exemplo: Vimos que esses parâmetros possuem padrões “default” quando são deixados em branco. Se não colocar o início, significa que desejamos começar do 0. Se não colocar o fim, significa que desejamos que o fatiamento termine no último caractere + 1 (ou seja, o último caractere da string fazendo parte do fatiamento). Se não colocarmos o passo, significa que desejamos que nada seja pulado. Os caracteres são pegos de 1 a 1.

Vamos testar!

```
1  exemplo = "Hello, World!"
2  print(exemplo) # string inteira
3  print(exemplo[0:13:1]) # informando início, fim e passo
4  print(exemplo[:13:1]) # informando fim e passo
5  print(exemplo[::1]) # informando somente o passo
6  print(exemplo[::]) # todos os parâmetros padrão
7  print(exemplo[:])  # é a mesma coisa que escrever exemplo[::]
8
```

```
Hello, World!
Hello, World!
Hello, World!
Hello, World!
Hello, World!
Hello, World!

Process finished with exit code 0
```

O resultado foi o mesmo para todos os casos! Conclusão: realizar esse tipo de fatiamento, seja indicando expressamente os parâmetros ou deixando eles em branco (padrão), equivale a simplesmente pegar a string inteira.

Segundo exemplo: Pegar pedaços da string de índice em índice (passo 1):

```
1    exemplo = "Hello, World!"
2
3    print(exemplo[7:13:1])
4
```

```
World!

Process finished with exit code 0
```

Aqui pegamos um pedaço que começou a partir do índice 7 (ele incluso) – "W" - e fomos até o último índice da string + 1 (assim incluímos o sinal de exclamação). Com isso pegamos só um pedaço da string.

Podemos pegar qualquer pedaço, conforme seja conveniente para nós. Também podemos nos valer da técnica de deixar alguns parâmetros em branco para aproveitarmos as configurações padrão, da mesma forma que demonstramos no primeiro exemplo.

```
World!
World
Hello
Hello
World!
llo, W
llo, W

Process finished with exit code 0
```

Além desses exemplos, saiba que após indicar o início e o fim do *slicing* nem é necessário inserir os : relativos ao parâmetro do passo, quando desejarmos que ele seja 1 (o padrão por definição).

Assim, quando quisermos o intervalo entre os índices 2 e 8 com o passo padrão (1), vamos simplesmente usar [2:8] e não [2:8:] (não que a segunda forma esteja incorreta, trata-se apenas da prática no dia a dia do programador).

Terceiro Exemplo: Alterando o passo.

Podemos usar passo diferente do padrão 1. Em vez de pegar caractere por caractere no intervalo podemos intercalar, por exemplo: pega um, pula o próximo, pega esse, pula o seguinte, etc.

Assim, podemos começar do índice 0 e ir pegando de 2 em 2, ou seja, 0, 2, 4, 6. Ou podemos começar do 3 e irmos pegando de 4 em 4, ou seja, 3, 7, 11.

Alterando o passo conseguimos alguns resultados bem peculiares:

```
exemplo = "Hello, World!"
print(exemplo[::2]) # string inteira de 2 em 2
print(exemplo[1::3]) # do índice 1 até o final (inclusivo) de 3 em 3
print(exemplo[3::4]) # do índice 3 até o final (inclusivo) de 4 em 4
print(exemplo[2:8:2]) # do índice 2 ao 8 (não incluí o 8) de 2 em 2

```

```
Hlo ol!
eoWl
lWd
lo

Process finished with exit code 0
```

Os pedaços ficaram bem confusos e estranhos. Por isso é necessário saber qual o passo é mais conveniente dependendo do pedaço e dos caracteres que precisamos acessar.

Mas a verdadeira utilidade do passo está em sua habilidade especial: com ele é possível inverter a string! Podemos ler a string do fim ao início!

Já estudamos que é possível usar índices negativos para acessar caracteres da string. -1 seria o último, -2 o penúltimo e assim sucessivamente. Da mesma forma, se passarmos -1 como parâmetro para o Python, o programa vai entender que desejamos que o nosso pedaço seja lido de trás para frente, invertendo a ordem natural de leitura!

Todo e qualquer valor negativo passado como parâmetro será entendido dessa maneira: -1 irá inverter a string percorrendo todos os caracteres. -2 irá inverter percorrendo de 2 em 2 (um sim e um não). Vamos mostrar como funciona:

```
exemplo = "Hello, World!"
print(exemplo[::-1]) # string inteira do início ao fim (incluído), mas INVERTIDA
print(exemplo[::-2]) # string inteira do início ao fim (incluído), mas INVERTIDA e de 2 em 2

```

```
!dlroW ,olleH
!lo olH

Process finished with exit code 0
```

Essa funcionalidade é muito útil e importante, facilitando a vida do programador em muitos casos! Recomendamos que você teste todos os parâmetros e situações que abordamos para que veja os diferentes resultados e compreenda o funcionamento do acesso de caracteres e do *slicing*, pois ambos são essenciais ao programador.

ATENÇÃO: Não existe passo 0! Isso não faria sentido, pois seria o mesmo que pedir ao programa para não fazer nada com o pedaço, ou seja, não pegar o pedaço. Se passarmos o parâmetro 0 para o passo vamos conseguir uma mensagem de erro de valor:

```
exemplo = "Hello, World!"
print(exemplo[::0])

```

```
Traceback (most recent call last):
  File "C:\Users\rafae\PycharmProjects\Tutorial\.venv\strings.py", line 2, in <module>
    print(exemplo[::0])
          ~~~~~~~~^^^^
ValueError: slice step cannot be zero

Process finished with exit code 1
```

1.4 Métodos de String

As strings possuem diversas funcionalidades que já estão programadas no Python, bastando ao programador invocá-las para fazer uso delas. São diversas funções, denominadas métodos, que realizam ações úteis sobre as strings, facilitando a vida do programador.

Na primeira parte deste tutorial falamos sobre alguns desses métodos, tais quais o strip() e o lower(). Relembrando: strip remove eventuais espaços em branco que venham no início e ao final da string, enquanto lower deixa todos os caracteres em letra minúscula.

Se esses métodos não existissem, nós mesmos é que teríamos de criá-los e implementá-los manualmente, o que tornaria a programação mais difícil, exigindo maior conhecimento técnico e gasto de tempo para programarmos essas funções.

Assim, as strings já possuem diversos métodos que irão nos ajudar. Vamos estudar os principais que farão parte do cotidiano do programador. Existem muitos métodos, porém todos são intuitivos e fáceis de usar!

ATENÇÃO: Somente iremos falar dos métodos mais importantes e mais utilizados no dia a dia do programador. Existem outros métodos além daqueles que trataremos nos itens seguintes. Para consultá-los basta pesquisar na documentação oficial do Python ou realizar buscas no google ou pedir ajuda para a inteligência artificial.

1.4.1 Capitalização – upper(), lower(), capitalize(), title() e swapcase()

Esses métodos permitem que manipulemos a capitalização das letras e palavras que compõem a string, modificando de maiúsculas para minúsculas ou vice-versa.

Vejamos:

a) O método upper() passa todas as letras para maiúsculas;
b) O método lower() passa todas as letras para minúsculas;
c) O método capitalize() deixa apenas a primeira letra da string em maiúscula e o resto fica minúsculo;
d) O método title() deixa a primeira letra de cada palavra da string em maiúscula;
e) O método swapcase() inverte maiúsculas e minúsculas – o que era maiúsculo vira minúsculo e o que estava em minúsculo passa a ser maiúsculo.

Vamos testar o funcionamento desses métodos. Para isso vamos usar como exemplo a string "Estudando métodos de capitalização em Python!":

```
1   teste = "Estudando métodos de capitalização em Python!"
2
3   maiuscula = teste.upper()
4   minuscula = teste.lower()
5   capitalizado = teste.capitalize()
6   titulo = teste.title()
7   trocado = teste.swapcase()
8
9
10  print(maiuscula)
11  print(minuscula)
12  print(capitalizado)
13  print(titulo)
14  print(trocado)
15
```

Resultado:

```
ESTUDANDO MÉTODOS DE CAPITALIZAÇÃO EM PYTHON!
estudando métodos de capitalização em python!
Estudando métodos de capitalização em python!
Estudando Métodos De Capitalização Em Python!
eSTUDANDO MÉTODOS DE CAPITALIZAÇÃO EM pYTHON!

Process finished with exit code 0
```

No caso optamos por salvar o resultado de cada método em uma variável nova para facilitar a compreensão, mas podemos printar diretamente o resultado da aplicação do método sem o atribuir a uma variável. Exemplo:

```
1    teste = "Estudando métodos de capitalização em Python!"
2
3    print(teste.upper())
```

```
ESTUDANDO METODOS DE CAPITALIZAÇAO EM PYTHON!

Process finished with exit code 0
```

Veja que já mandamos printar direto o texto após usar o método upper(), sem haver necessidade de usar a variável de nome “maiuscula” que criamos no exemplo anterior.

ATENÇÃO: Esses métodos NÃO alteram a string original. O princípio da imutabilidade permanece. Eles apenas retornam uma nova string. Vamos provar isso printando a string original após usar um dos métodos:

```
1    imutabilidade = "EU NÃO VOU MUDAR!"
2
3    print(imutabilidade.lower())
4    print(imutabilidade)
5
```

```
eu não vou mudar!
EU NÃO VOU MUDAR!

Process finished with exit code 0
```

Como podemos ver, a linha 3 retornou uma string com todas as letras em minúsculas. Porém, quando mandamos printar a string orginal na linha 4, ela permanece em maiúsculas e imutável. Ou seja: **provamos que o método retornou uma NOVA string e não modificou a string original!**

1.4.2 Análise de Conteúdo – isalpha(), isdigit(), isalnum(), isspace()

Esses métodos servem para analisar o tipo dos caracteres que compõem a string. Isso é muito útil para validar dados recebidos do usuário e para garantir que o conteúdo digitado está dentro do esperado — por exemplo, conferir se o usuário digitou apenas letras, apenas números ou se deixou o campo em branco.

Métodos de análise de conteúdo sempre retornam um valor booleano – True ou False. Vejamos:

a) O método isalpha() retorna True se todos os caracteres forem letras (sem espaços, números ou símbolos);
b) O método isdigit() retorna True se todos os caracteres forem números (algarismos de 0 a 9);
c) O método isalnum() retorna True se todos os caracteres forem alfanuméricos (letras e números, mas sem espaços nem símbolos);
d) O método isspace() retorna True se a string for composta somente por espaços vazios (“ “);

Vamos testar esses métodos com uma string composta por letras e números. teste = “Apollo13”:

```
1   teste = "Apollo13"
2
3   print(teste.isalpha()) # False, pois tem algarismos na string
4   print(teste.isdigit()) # False, pois tem letras na string
5   print(teste.isalnum()) # True, pois a string é composta por letras e algarismos
6   print(teste.isspace()) # False, pois a string não é composta somente de espaços vazios
7
```

Console:

```
False
False
True
False

Process finished with exit code 0
```

Recomendamos que o leitor fique brincando com esses métodos e testando diferentes strings para entender o comportamento de cada uma dessas funcionalidades, pois elas são úteis para o tratamento dos dados fornecidos pelo usuário.

1.4.3 Busca e Contagem – find(), count(), startswith(), endswith()

Esses métodos permitem buscar e contar trechos específicos de uma *string*, que podem ser tanto um único caractere quanto uma sequência de caracteres. Esse tipo de trecho é chamado de *substring*, pois corresponde a uma *string* menor contida dentro de uma *string* maior.

Por exemplo, considerando a string “banana”, podemos dizer que “ban” é uma substring dessa string principal, pois “ban” faz parte da palavra (string) “banana”.

Vejamos os principais métodos:

a) O método find() retorna o índice de ínicio do trecho que buscamos. No exemplo da buscar “ban” em “banana”, ele retornaria 0, pois a substring “ban” já começa da letra “b” da palavra “banana” que está na posição 0! Caso busquemos apenas 1 caractere, ele irá retornar a posição da PRIMEIRA ocorrência desse caractere. Na palavra “banana”, se buscarmos a letra “a”, o método find() vai nos dizer que essa letra está na posição 1 e as demais letras “a” da string serão ignoradas. **ATENÇÃO:** Caso o find() não encontre o caractere ou a expressão buscada, ele vai exibir “-1” quando for printado ou armazenado em uma variável;
b) O método count() conta quantas vezes o caractere ou o trecho aparece na string. Caso não apareça nenhuma vez, o resultado será 0;
c) O método startswith() retorna True se a string começar com o caractere ou o trecho indicado;
d) O método endswith() retorna True se a string terminar com o caractere ou o trecho indicado.

Mais uma vez, vamos ver o funcionamento desses métodos na prática:

```
teste = "banana"

print(teste.find("ban")) # 0, pois a substring começa na posição 0
print(teste.find("n")) # 2, pois o primeiro "n" encontrado está na posição 2
print(teste.find("d")) # -1, pois não há o caractere "d" na string "banana"
print(teste.count("a")) # 3, pois a letra "a" aparece 3 vezes na string
print(teste.startswith("b")) # True, pois "banana" começa com "b"
print(teste.endswith("na")) # True, pois "banana" termina com "na"

```

Resultado no console:

```
0
2
-1
3
True
True

Process finished with exit code 0
```

1.4.4 Modificação da String

Esses métodos geram novas versões da string, permitindo que removamos espaços, ou realizemos a substituição de trechos da string.

Vamos lembrar que, apesar de chamarmos esses métodos de "métodos de modificação da string", as **strings são imutáveis**. Essas funcionalidades não modificam a string original, mas sim nos retornam uma nova string com as alterações desejadas.

Estudaremos os seguintes métodos:

a) O método replace() permite que troquemos um trecho da string por outro trecho. Por exemplo, podemos remover os espaços mandando substituí-los por um espaço vazio, ou fazer com que as letras "a" de determinada palavra virem a letra "e".

A sintaxe desse método é: replace(antigo, novo, contador(opcional))

Substituindo “a” por “e” ficaria algo assim: string.replace(“a”, “b”). Nesse exemplo TODAS as letras “a” que estejam na string serão substituídas pela letra “e”.

Mas e se quisermos substituir apenas a primeira letra “a”? É para isso que serve o terceiro parâmetro – o contador (que é opcional). Nesse nosso exemplo usaríamos a seguinte sintaxe: string.replace(“a”, “e”, 1). Isso significa que somente a primeira letra “a” encontrada seria substituída. O replace somente seria aplicado na primeira ocorrência.

b) Os métodos strip(), lstrip() e rstrip() servem para remover caracteres que venham antes e depois da string. Todos esses métodos têm por padrão a remoção de espaços, o que significa que se não passarmos nenhum parâmetro para esses métodos, eles vão entender que o programador deseja remover os espaços que venham antes e/ou depois da string.

   O strip() remove os caracteres do início e do fim da string, já o lstrip() somente os do início (l de left – esquerda) e o rstrip() somente os que estão no final (r de right – direita)

c) Os métodos center(), ljust() e rjust() visam alinhar o texto em um espaço específico. Podemos alinhar usando espaços em brancos ou determinar que seja usado um caractere – exemplo: string.center(20, “-“). Nesse exemplo o texto vai ficar centralizado com os símbolos “-“ aparecendo antes e depois dele até preencherem 20 caracteres (considerando o texto também). Se passarmos somente um número e mais nenhum parâmetro, o método vai entender que queremos utilizar espaços em branco.

   Já o ljust() e o rjust() seguem a mesma lógica, mas em vez de centralizar, vão aplicar os espaços ou caracteres, respectivamente, no fim ou no início da string.

Vamos ver esses métodos funcionando na prática:

```
1  exemplo = "   Vamos aprender métodos de strings!   "
2
3  print(exemplo.replace( _old: " ", _new: ""))
4  print(exemplo.replace( _old: "e", _new: "f", _count: 2))
5  print(exemplo.strip())
6  print(exemplo.lstrip())
7  print(exemplo.center(50))
8  print(exemplo.ljust( _width: 10, _fillchar: "-"))
9  print(exemplo.rjust( _width: 100, _fillchar: "-"))
```

Console:

```
Vamosaprendermétodosdestrings!
   Vamos aprfndfr métodos de strings!
Vamos aprender métodos de strings!
Vamos aprender métodos de strings!
       Vamos aprender métodos de strings!
   Vamos aprender métodos de strings!
----------------------------------------------------------   Vamos aprender métodos de strings!
```

Atenção para as linhas 8 e 9 e os seus respectivos resultados no console. Observem que a linha 8 desejava que o texto fosse ajustado para a esquerda com a adição de 10 “-“ no final, mas isso não aconteceu. Por quê?

Os métodos center(), ljust() e rjust() entendem o primeiro parâmetro como o tamanho total da string. A nossa string original ocupava mais de 10 espaços (tinha mais de 10 caracteres), então quando usamos o ljust(10, “-“), não tinha como adicionar mais tracinhos ao final, pois já tínhamos mais de 10 caracteres no texto.

Por outro lado, o rjust(100, “-“) conseguiu jogar o texto para a direita e adicionar os tracinhos antes, fazendo com que a soma de tracinhos e a string original atingisse o tamanho 100 que passamos como parâmetro!

1.4.5 Métodos join() e split()

Esses métodos são muito importantes, mas não poderão ser estudados agora, pois precisamos ter conhecimentos mais aprofundados sobre as estruturas de dados e iteráveis, que serão estudadas na parte III desse tutorial.

Porém, por serem muito importantes, é interessante que já saibamos que eles existem e que futuramente vamos aprender como eles funcionam.

O método join() é utilizado para unir e concatenar diversas strings que estejam contidas em uma estrutura iterável (como uma lista ou uma tupla). Quando estudarmos essas estruturas de dados vamos voltar nesse tópico. Apenas entenda que existe esse método e que ele é importante para a programação em Python.

Por outro lado, o método split() faz o oposto: ele separa o conteúdo de uma string em partes, colocando cada parte dentro de uma lista. O programador é que define qual será o separador dessas partes (por exemplo, se escolhermos o espaço em branco como separador, teremos uma lista composta pelas palavras que compõem o texto da string). Novamente, não se preocupe. No futuro isso vai ser estudado e fará sentido.

No momento somente estamos tendo esse contato inicial com esses métodos.

1.5 Função len()

A função len() não é um método de string, mas sim uma função que pode ser utilizada com iteráveis para nos indicar o seu tamanho. Como as strings são iteráveis (afinal são estruturas de dados sequenciais), podemos usar a função len() para saber o seu tamanho exato.

A sua sintaxe é: len(string).

Essa função irá os retornar um número inteiro que indica o tamanho do iterável. No caso da string, teremos o número de caracteres que compõem o texto.

```
1    teste = "Qual o meu tamanho?"
2
3    print(len(teste))
4
```

Resultado:

```
19

Process finished with exit code 0
```

Isso significa que a string teste tem 19 caracteres. O seu tamanho é 19!

Essa função é muito simples, mas também é muito importante e faz parte do dia a dia do programador.

CAPÍTULO 2 – Exercícios com Strings

2.1 Introdução

O domínio do conceito e da manipulação de strings é essencial ao programador. Por ser um assunto relativamente extenso e moderadamente complexo, inclusive com alguns pontos que somente iremos compreender completamente quando estudarmos tópicos mais avançados, vamos resolver alguns exercícios para fixar os conceitos estudados.

2.2 Slicing – Frutas em Ordem Alfabética

Vamos criar um programa simples que vai pegar uma string contendo várias frutas. A string será a seguinte:

frutas = “kiwi, pera, banana, abacaxi, laranja”

O objetivo do exercício será printar no console as frutas em ordem alfabética, usando o conhecimento de slicing (fatiamento) que estudamos. Tente criar esse programa antes de ver a resolução!

Resolução

Para criarmos o nosso programa, primeiro temos de contar os caracteres da string “frutas” para saber os intervalos de cada palavra na string (os índices que ocupam). Lembrando que os espaços em branco e a vírgula também são caracteres!

Então, “kiwi” ocupa desde o 0 até o índice 3 da string. Sabemos que para fazer o fatiamento temos de pegar o intervalo que a fatia que queremos está. Como o intervalo sempre excluí o último índice, temos que pegar do 0 ao 4 para cortar a palavra “kiwi” da nossa string.

Assim, o slicing de “kiwi” seria frutas[0:4]. Basta printar isso para aparecer “kiwi” no console.

Vamos usar essa mesma lógica para todas as frutas e printá-las em ordem alfabética, para serem exibidas dessa forma no console. Veja como o programa vai ficar:

```
frutas = "kiwi, pera, banana, abacaxi, laranja"

print(frutas[20:27])
print(frutas[12:18])
print(frutas[0:4])
print(frutas[29:37])
print(frutas[6:10])

```

Executando o programa:

```
abacaxi
banana
kiwi
laranja
pera

Process finished with exit code 0
```

Pronto! Todas as frutas foram exibidas em ordem alfabética. Também podemos usar as funcionalidades da função print() que estudamos na parte 1 do tutorial para exibir todos na mesma linha e separados por vírgula, usando o parâmetro end:

```
frutas = "kiwi, pera, banana, abacaxi, laranja"

print(frutas[20:27], end=", ")
print(frutas[12:18], end=", ")
print(frutas[0:4], end=", ")
print(frutas[29:37], end=", ")
print(frutas[6:10])

```

```
abacaxi, banana, kiwi, laranja, pera

Process finished with exit code 0
```

Lembrando que o **end=", "** indicou para o **print()** que não queremos uma quebra de linha ao final, mas sim que seja adicionada uma vírgula seguida de um espaço. Isso permite que as frutas sejam exibidas separadas por vírgula e na mesma linha. No último print não precisamos modificar o end, pois não vem nada depois dele.

2.3 Analisando a String

Nesse exercício vamos criar um programa que recebe input do usuário e analisa o que ele digitou, printando no console o resultado dessa análise.

O programa deve analisar os seguintes itens:

a) Averiguar se a string é composta apenas por letras (sem números, espaços ou símbolos);
b) Averiguar se a string é composta apenas por números;
c) Averiguar se a string é composta por caracteres alfanuméricos (somente letras e números);
d) Averiguar se a string é composta apenas por espaços vazios.

Lembrando que o input do usuário pode variar. Para fins do nosso programa, vamos usar a string "Apollo13" como input do usuário, mas é interessante que usemos vários inputs diferentes para vermos o comportamento da nossa análise sob diferentes condições.

Resolva o problema antes de ver a solução! Entregue o resultado da análise ao usuário de forma intuitiva e fácil de entender.

Resolução

Temos de criar um programa que receba o input do usuário. Para isso temos a função input(). Quanto à análise da string, podemos usar os métodos de string que estudamos para resolver esse problema. Também é conveniente usarmos estruturas de decisão (**if-else**) para controlar qual mensagem será exibida ao usuário.

Vamos escrever o código!

```
texto = input("Digite uma string a ser analisada: ")
print("Resultado da análise:")

if texto.isalpha():
    print("A string analisada é composta apenas por letras.")
else:
    print("A string analisada não é composta somente por letras.")
if texto.isdigit():
    print("A string analisada é composta apenas por algarismos.")
else:
    print("A string analisada não é composta somente por algarismos.")
if texto.isalnum():
    print("A string analisada é composta apenas por letras e números.")
else:
    print("A string analisada não é composta apenas por caracteres alfanuméricos.")
if texto.isspace():
    print("A string analisada é composta somente por espaços vazios.")
else:
    print("A string analisada não é composta apenas por espaços vazios.")

```

Executando:

```
Digite uma string a ser analisada: Apollo13
Resultado da análise:
A string analisada não é composta somente por letras.
A string analisada não é composta somente por algarismos.
A string analisada é composta apenas por letras e números.
A string analisada não é composta apenas por espaços vazios.

Process finished with exit code 0
```

Recomendamos testar diversos outros inputs! Vamos analisar um pouco mais a fundo a nossa resolução.

Por que usar vários if-else em vez de elif? Conforme já estudamos na parte 1, usamos vários if quando queremos que todas as nossas condicionais sejam avaliadas pelo programa. A estrutura elif não permite isso. Se usarmos elif, assim que o Python encontrar a primeira condição satisfeita ele vai ignorar as demais. **Elas nem serão analisadas!**

Então, nesse caso, para que o programa funcione corretamente temos de usar if!

Esses métodos não retornam True ou False? Por que não tivemos de verificar se cada método era True ou False para as condicionais? Nós verificamos! Quando escrevemos, por exemplo, **if texto.isalpha()** estamos dizendo para o Python que a condicional é o resultado de texto.isalpha() ser True.

Escrever isso é o mesmo que escrever **if texto.isalpha() == True** (comparação expressa). Mas por que isso funciona? Simples! Se algo é (se algo existe) então ele é True! Contrariamente, se algo não é (não existe) então ele é False. Não precisamos escrever a comparação usando igualdade.

Se quiséssemos inverter a lógica da condicional para fazer algo somente se isalpha() fosse False, bastaria escrever **if not texto.isalpha()**. Isso é o mesmo que escrever **if texto.isalpha() == False**!

ATENÇÃO: Os programadores evitam essa redundância na comparação. Sempre usamos a forma mais limpa e considerada natural (a forma que usamos na resolução do nosso exercício).

2.4 Detectando Palíndromos

Agora vamos criar um programa que recebe um input do usuário que pode ser uma palavra ou uma frase. O nosso programa irá identificar se o que o usuário digitou é um palíndromo.

Palíndromos são palavras ou textos que podem ser lidos de trás para frente. A palavra “ama” ou a palavra “osso” podem ser lidos de trás para frente sem problemas que resultam na mesma palavra!

Vamos escrever um código que faça isso! Crie o seu código e depois vamos conferir a resolução!

Resolução

Para escrever o código vamos precisar seguir os seguintes passos:

1 – receber uma string do usuário;

2 – inverter essa string e salvar em uma nova variável;

3 – comparar a string original com a string invertida;

4 – programar as respostas para caso seja detectado (ou não) um palíndromo.

Vamos ver funcionando:

```
texto = input("Escreva uma palavra: ")
texto_invertido = texto[::-1]

if texto == texto_invertido:
    print("A palavra digitada é um palíndromo!")
else:
    print("A palavra digitada não é um palíndromo!")

```

Vamos supor que o usuário desejou testar a palavra “osso”. Segue o resultado:

```
Escreva uma palavra: osso
A palavra digitada é um palíndromo!

Process finished with exit code 0
```

Esse programa funcionou corretamente, mas ainda está muito básico e pode apresentar problemas dependendo do input do usuário. É possível que ele digite uma palavra que é um palíndromo, mas o nosso programa não consiga perceber isso.

Por exemplo, se o usuário digitasse "Osso" em vez de "osso", quando invertermos ao texto o resultado seria "ossO". **E, para o Python, "Osso" é diferente de "ossO", pois maiúsculas e minúsculas são diferenciadas!**

Vamos testar na prática:

```
Escreva uma palavra: Osso
A palavra digitada não é um palíndromo!

Process finished with exit code 0
```

Realmente, uma simples letra maiúscula quebrou o funcionamento do nosso programa. Felizmente, já estudamos formas de realizar um tratamento de dados básico que pode melhorar a robustez do nosso código. Vamos aprimorar esse programa!

2.4.1 Melhorando o Programa – Tratamento de Dados e Análise de Frases

Não são apenas as letras maiúsculas e minúsculas que podem nos causar problemas. Espaços também podem ser problemáticos, pois o Python também irá considerá-los quando for comparar as strings.

Então, o usuário digitar um espaço no início ou no fim da palavra por engano (vamos supor que o dedo dele escorregou ao digitar e acertou a tecla de espaço sem querer), o programa também não irá funcionar corretamente.

Um caminho para resolver esses problemas seria garantir que tudo ficasse em letras minúsculas e que eventuais espaços antes e depois da string fossem removidos. Conhecemos os métodos de strings que podem nos ajudar: **lower()** e **strip()**!

Vamos implementá-lo e ver se o programa consegue lidar com o input “ Osso” (“O” maiúsculo e um espaço em branco antes da palavra):

```
texto = input("Escreva uma palavra: ").lower().strip()
texto_invertido = texto[::-1]

if texto == texto_invertido:
    print("A palavra digitada é um palíndromo!")
else:
    print("A palavra digitada não é um palíndromo!")

```

Console:

```
Escreva uma palavra:  Osso
A palavra digitada é um palíndromo!

Process finished with exit code 0
```

Conseguimos! Nosso programa se tornou mais robusto com um simples tratamento básico de dados.

Podemos, agora, preparar nosso programa para receber frases inteiras e checar se elas também são palíndromas! A frase palíndroma segue a mesma lógica da palavra: pode ser lida de trás para frente. Exemplo: “Anotaram a data da maratona”.

Vamos ver se funciona?

```
Escreva uma palavra: Anotaram a data da maratona
A palavra digitada não é um palíndromo!

Process finished with exit code 0
```

Parece que o nosso programa ainda não está pronto para lidar com frases. O que aconteceu?

Se analisarmos bem a frase, veremos que ao invertê-la os espaços não coincidem. Então vamos precisar de um tratamento de dados diferenciado que contemple não somente os espaços no início e no fim da string, mas os que estão no seu meio separando as palavras também!

Mas como remover esses espaços? strip() não consegue fazer isso. Por sorte temos o método replace() e nós já estudamos ele! Com esse método conseguimos remover todos os espaços, tanto os do início e final da string quanto os do meio.

Vamos ver se o **replace()** irá nos ajudar:

```
texto = input("Escreva uma palavra: ").lower().replace( _old: " ", _new: "")
texto_invertido = texto[::-1]

if texto == texto_invertido:
    print("A palavra digitada é um palíndromo!")
else:
    print("A palavra digitada não é um palíndromo!")

```

```
Escreva uma palavra: Anotaram a data da maratona
A palavra digitada é um palíndromo!

Process finished with exit code 0
```

Agora sim! Deu certo! Como o **replace()** está trocando todos os espaços, nem precisamos mais usar o **strip()**. O nosso programa agora vai conseguir lidar bem melhor com as palavras e também conseguirá analisar as frases!

CAPÍTULO 3 – Estruturas de Repetição: Laço For

3.1 Introdução aos Laços de Repetição

No desenvolvimento de algum programa é comum termos blocos de código que acabam se repetindo várias vezes no código. É justamente para lidar com essa questão que as estruturas de repetição existem.

Essas estruturas, após serem montadas, permitem que o trecho do código seja repetido por um número determinado de vezes ou até que uma condição seja satisfeita. Isso facilita em muito o trabalho do desenvolvedor, além de tornar o código mais limpo e mais fácil para a manutenção.

Podemos montar essas estruturas usando dois tipos de laços diferentes: for e while. Nesse capítulo estudaremos o funcionamento do laço for e, no próximo, falaremos sobre o while.

3.2 Laço For – For Loop

Esse laço permite que o Python percorra os elementos de uma estrutura iterável. Se você se lembra bem, quando estudamos strings foi mencionado que uma das características desse tipo básico é ser um iterável.

E o que significa ser iterável? Significa que é uma estrutura que pode ser percorrida por um laço for elemento a elemento. Isso quer dizer que cada elemento que compõe a string pode ser acessado individualmente e de forma automática pelo laço for.

Cada loop do for, ou seja, cada repetição é chamada de iteração.

Vamos dar um exemplo! Pense que você está desenvolvendo um programa que deve soletrar uma palavra. O usuário quer que a palavra “anticonstitucional” seja soletrada.

Com os nossos conhecimentos de fatiamento de strings (slicing), nós podemos resolver esse problema acessando os índices individuais de cada letra e mandando printá-los. Já fizemos isso nos exercícios do capítulo anterior!

Só que essa palavra – “anticonstitucional” é muito grande. Ela tem 18 letras, ou seja, teríamos de acessar 18 itens diferentes para printar no console as letras individualmente, soletrando a palavra.

Esse é um clássico exemplo de aplicação do laço for. Essa estrutura de repetição irá nos poupar de fazer isso tudo, automatizando esse processo. Vamos estudar como isso é feito.

3.3 Sintaxe do For

Quando desejamos implementar um for loop (laço for), usamos a seguinte sintaxe:

for variável in sequência:

    instruções

Então temos de usar a palavra reservada for, atribuir uma variável, usar o operador in e indicar a sequência a ser iterada. Em seguida usamos os : e, na próxima linha, passamos as instruções com identação (lembra da estrutura if-else? Na parte 1 desse tutorial já vimos isso).

Vamos criar o nosso programa simples que vai soletrar uma palavra para ver o for em funcionamento:

```
palavra = "carro"

for letra in palavra:
    print(letra)

```

Criamos uma variável “palavra” com a palavra a ser soletrada. No loop for usamos a sintaxe que estudamos: palavra reservada for, seguida da variável de controle “letra” criada por nós que vai percorrer o iterável (no caso uma string) “palavra”.

Como instrução, mandamos printar o conteúdo da variável “letra” em cada iteração (em cada repetição). Como “letra” vai acessando de 1 a 1 o conteúdo da string “palavra”, a cada repetição teremos uma letra diferente dentro de “letra”. Vamos executar esse programa e ver o resultado no console:

```
c
a
r
r
o

Process finished with exit code 0
```

Então, na primeira iteração, “letra” pegou o “c” e foi printado. Na segunda, o “c” foi substituído pelo “a” e foi printado e assim sucessivamente até passar por todos os elementos que compõem a string.

ATENÇÃO: O **laço for** somente consegue percorrer iteráveis, ou seja, sequências. Caso tentemos percorrer algo que não possui essa característica, vamos quebrar o programa e receber uma mensagem de erro.

Vamos mostrar um exemplo de bom uso do for. Para isso vamos criar uma string com números e vamos printar todos os algarismos que compõem esse número no console:

```
numero = "12345"

for algarismo in numero:
    print(algarismo)

```

```
1
2
3
4
5

Process finished with exit code 0
```

O programa funcionou corretamente, pois a variável “numero” é uma string e strings são iteráveis!

Agora o que aconteceria se a variável “numero” fosse um int? Quando estudamos esse tipo na parte I do tutorial não falamos que ele é um iterável como fizemos com as strings. Será que funciona? Vamos testar:

```
numero = 12345

for algarismo in numero:
    print(algarismo)

```

```
Traceback (most recent call last):
  File "C:\Users\rafae\PycharmProjects\Tutorial\.venv\for_loop.py", line 3, in <module>
    for algarismo in numero:
                     ^^^^^^
TypeError: 'int' object is not iterable

Process finished with exit code 1
```

Mensagem de erro! Veja que o Python está indicando que o problema está na variável “numero”. Essa variável é um int e, **int não é iterável!** Para o Python, um int não é uma sequência e, portanto, não podemos usar um laço for para percorrer os seus elementos.

3.4 Função range()

Percorrer e acessar os elementos individuais de determinado iterável um a um é muito útil quando queremos expor esses elementos. Agora, quando o objetivo do nosso loop for apenas repetir um trecho de código um determinado número de vezes (ou seja, sabemos exatamente o número de repetições que queremos) podemos fazer uso da função range.

Essa função irá nos dar um intervalo de valores que podemos usar no loop for. O loop irá percorrer esse intervalo e isso nos dará o número de repetições desejado.

A função range funciona de forma muito parecida com o slicing. Veja: range(início, fim, passo). Início é o número de início do intervalo, fim é o número final desse intervalo **(que não fará parte dele!)** e passo é a forma de contagem do intervalo.

ATENÇÃO: A função range() pega os números que estão entre o intervalo, considerando o do início (incluí o início) e não incluindo o fim **(o último número NÃO aparece)**. Se o programador pegar o range entre 1 e 10, ele selecionou os números que estão entre 1 (incluído) e 10 (excluído), ou seja, números de 1 a 9!

A range() possui valores padrão, ou seja, não precisamos passar todos os parâmetros de início, fim e passo para ela. Por exemplo, se digitarmos apenas range(10), a função irá nos entregar os números que estão entre 0 (início) e 10 (fim), de 1 em 1, ou seja, de 0 a 9 totalizando 10 números.

Então temos três maneiras diferentes de passar parâmetros para a função range:

a) Completo = início, fim e passo = range(0,10,1);
b) Somente um número = vai de 0 até o número = range(10);
c) Somente passando início e fim = o passo por padrão é de 1 em 1 = range(0,10).

ATENÇÃO: Não podemos passar somente um valor como intervalo desejado e outro como passo, pois isso vai confundir a função e vai dar erro. Se quisermos todos os números pares do intervalo 10, não podemos simplesmente escrever range(10,2), pois a função não vai saber se esse 2 é relativo ao passo ou ao fim do intervalo!

Vamos mostrar o funcionamento do range() na prática.

3.4.1 Usando range() Para Contar Repetições

Esse é o uso mais comum da função. Temos um bloco de código que queremos que seja repetido por um número conhecido de vezes. Vamos supor que desejamos que a mensagem "Gosto muito de você!" seja exibida 5 vezes seguidas no console. Podemos fazer usar o laço for junto com a função range(5):

```
1    for _ in range(5):
2        print("Gosto muito de você!")
3
```

Executando:

```
Gosto muito de você!
Gosto muito de você!
Gosto muito de você!
Gosto muito de você!
Gosto muito de você!

Process finished with exit code 0
```

Então não precisamos escrever 5 linhas de código iguais determinando o print da mensagem. Usamos o intervalo que está de 0 a 5 -> 0, 1, 2, 3, 4 = 5 iterações para repetir a instrução que mandou printar a frase.

Então, temos 5 valores nesse intervalo, o que fará com que a instrução print(“Gosto muito de você!”) seja repetida 5 vezes.

3.4.2 Usando range() Para Exibir Números

Assim como fizemos no exemplo da palavra soletrada, também podemos expor os números que estão no intervalo do range() no console. Vamos supor que queremos que sejam exibidos todos os números ímpares do intervalo de 1 a 12:

```
1    for i in range(1,12,2):
2        print(i)
3
```

Como conhecemos o funcionamento da **range()**, sabemos que isso vai funcionar. Vamos começar do 1 (que vai aparecer) e usamos passo 2 (ou seja vai percorrer de 2 em 2). Como o fim (número 12) não é incluído, o último número que vai aparecer é o 11. Vamos executar o programa:

```
1
3
5
7
9
11

Process finished with exit code 0
```

Se quiséssemos todos os números pares do intervalo, teríamos de começar de um número par em vez de um ímpar, mantendo o intervalo de 2 em 2. Porém, se o último número do intervalo também for par e desejamos que ele apareça, temos que adicionar +1 no fim desse intervalo.

Vamos fazer a mesma coisa do exemplo anterior: números pares do intervalo de 0 a 12, mas como desejamos que o 12 também apareça, terá de ser no intervalo de 0 a 13!

```
for i in range(0,13,2):
    print(i)

```

```
0
2
4
6
8
10
12

Process finished with exit code 0
```

Também podemos usar passo -1 para inverter a contagem. Nesse caso a função fica bem diferente: o parâmetro passado de início é maior que o parâmetro passado como fim. Vamos pensar numa contagem regressiva do 5 ao 0:

Vamos ter um intervalo que começa no 5, termina no 0 e vai de 1 em 1. Então range(5,0,-1). O -1 indica que queremos que seja de trás para frente. Será que vai funcionar?

```
for i in range(5,0,-1):
    print(i)

```

```
5
4
3
2
1

Process finished with exit code 0
```

Por que o 0 não apareceu? Lembre-se que o fim do intervalo sempre é excluído. Não importa se a sequência é crescente ou decrescente. Para que o 0 apareça, precisamos que o fim seja menor que 0. Então -1!

```
for i in range(5,-1,-1):
    print(i)

```

```
5
4
3
2
1
0

Process finished with exit code 0
```

Agora nossa contagem regressiva funcionou! Então lembrando: sequência crescente e queremos que o último número do intervalo apareça: +1 ao fim. Sequência decrescente e queremos que o último número do intervalo apareça: -1 ao fim.

3.4.3 Nome das Variáveis de Controle no range() - Convenções

Quando usamos a função range() no nosso loop for, temos a sintaxe:

for variável in range(x):

instruções

Essa variável pode ter qualquer nome, pois o programador é que decide como ela será nomeada. Recomenda-se que seja dado nome que faça sentido e que ajude na compreensão e manutenção do código.

No exemplo do programa que soletrava a palavra nós chamamos a variável de "letra":

```
palavra = "carro"

for letra in palavra:
    print(letra)

```

Entretanto, no exemplo da repetição dos prints usamos "_" como nome da variável:

```
for _ in range(5):
    print("Gosto muito de você!")

```

E no exemplo da contagem regressiva nós utilizamos "i":

```
for i in range(5,-1,-1):
    print(i)

```

Por qual razão fizemos isso? Vamos entender.

Realmente, o mais recomendável é que usemos nomes de variáveis que nos sejam úteis. Porém, usar "i" e "_" são consensos na programação.

ATENÇÃO: Repetimos: não é obrigatório usar "i" e "_". Somente estamos explicando que é comum o programador se deparar com loops for que utilizem esses nomes de variáveis.

Usamos "i" quando queremos uma variável que irá armazenar o valor para utilizá-lo depois. No caso da contagem regressiva, cada valor da iteração (de 5 até 0) era importante para ser armazenado, pois ele teria de ser printado no console. O mesmo

aconteceu no caso da palavra soletrada. Foi necessário armazenar cada letra em cada iteração para que ela pudesse ser exibida no console.

ATENÇÃO: Também é muito comum ver "j" e "k" além do "i" para esses casos (geralmente quando tempos mais de um loop for, conforme estudaremos adiante).

Usamos "_" quando queremos uma variável que não será usada depois. Ela somente existe para fazer a iteração do loop for. Foi o caso de printar a frase 5 vezes seguidas. Somente usamos o loop para não ter de repetir o código, mas não precisamos do conteúdo dessas iterações.

3.4.4 A range() Aceita Variáveis Numéricas

Conforme já demonstramos, temos de passar números como parâmetros para a função range() para indicar qual será o início do intervalo, o seu fim e o passo (sendo que podemos não passar início nem passo, pois eles possuem valores padrão como 0 e 1, respectivamente).

Então, sempre temos que passar números para essa função. Mas e se passarmos uma variável que recebeu um número como valor? Por exemplo, inicializamos a variável "numero" recebendo 3. Podemos passar "numero" como um dos parâmetros da range()?

Vamos ver se isso funciona!

```
1   numero = 3
2
3   for i in range(numero):
4       print(i)
5
```

```
0
1
2

Process finished with exit code 0
```

Funcionou! Ter passado a variável “numero” como parâmetro foi a mesma coisa que passar o seu conteúdo, no caso o número 3.

Isso é uma característica importante da programação que já estudamos na parte I do tutorial: as variáveis armazenam valores! Passar o valor diretamente ou passar uma variável que contenha esse mesmo valor é a mesma coisa!

Isso funciona até mesmo com o resultado de funções. Quando falamos de strings, nós estudamos a função **len()** que nos retorna o tamanho da string, ou seja, quantos índices ela tem.

Podemos passar o tamanho de uma string como parâmetro do range() usando essa função. Assim não temos de nos preocupar em contar quantos índices a string tem (e em alguns casos podemos nem ter como contar).

Veja o exemplo:

```
1    texto = "Quantos caracteres em um texto?"
2
3    for i in range(len(texto)):
4        print(i, end=" ")
```

```
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
Process finished with exit code 0
```

Como a frase tinha muitos caracteres (31), usamos o parâmetro end dentro da função print() para que os índices fossem exibidos na mesma linha. Senão não teria como colocar o resultado do console no material.

Veja como funcionou! A variável “texto” é uma string com 31 caracteres. O resultado de len(texto) é 31. Quando passamos essa função dentro do range, foi o mesmo que escrever range(31)! Esse tipo de técnica é útil e bastante utilizado.

3.5 Laço For Aninhado – Nested For Loop

É possível ter um laço for dentro de outro laço for. O Python permite fazer uso de estruturas aninhadas de for loop (como fizemos com if-else na parte I do tutorial). Em teoria, podemos ter tantos laços for aninhados quanto quisermos, mas isso pode aumentar a complexidade do programa, tornando mais difícil de entender o que ele faz e ocasionando lentidão.

Geralmente não ultrapassamos de 2 ou 3 laços for um dentro do outro. A sintaxe permanece a mesma, mas temos de tomar cuidado com as identações para que os blocos de código fiquem corretos. Vamos ver como fica:

a) Com 2 for:

```
for variável in sequência:
        for variável2 in sequência:
                instruções
```

b) Com 3 for:

```
for variável in sequência:
        for variável2 in sequência:
                for variável3 in sequência:
                        instruções
```

Essas estruturas permitem que façamos um for dentro de outro for. Há muitas situações em que isso é necessário, tais como gerar padrões, criar tabuleiros para jogos, criar matrizes, etc.

Vamos dar um exemplo criando uma matriz 2x2 usando 2 loops for. Assim conseguiremos explicar melhor o funcionamento dessa estrutura.

```
for i in range(2):
    for j in range(2):
        print(i, j)
```

```
0 0
0 1
1 0
1 1

Process finished with exit code 0
```

Então, a variável "i" está no range de 2 (ou seja, entre 0 e 2 sem incluir o 2) e a variável "j" também está no range de 2 (mais uma vez, entre 0 e 2 sem incluir o 2).

A instruções que passamos são para imprimir na mesma linha "i" e "j" ao fim de cada iteração. A tabela abaixo mostra como essas iterações funcionam:

Iteração	Valor de i	Valor de j
1ª	0	0
2ª	0	1
3ª	1	0
4ª	1	1

Esse exemplo foi equivalente a uma matriz 2x2. Cada linha do print exibido no console representa um par ordenado da matriz:

COLUNAS

LINHAS (0,0) (0,1)

(1,0) (,1,1)

Em ordem: linha 0 com coluna 0, linha 0 com coluna 1.

linha 1 com coluna 0, linha 1 com coluna 1.

Vamos entender melhor como funcionou o loop nesse caso. A tabela irá nos ajudar. Na primeira interação "i" deverá percorrer o range de 2, ou seja, 0 e 1 (sem o 2). Mas dentro do bloco de instruções do primeiro loop tem um outro loop para o "j"! Então o Python organiza as iterações da seguinte maneira:

Primeiro "i" assume o primeiro valor da iteração dele – 0 no caso – depois, as iterações do segundo loop são executadas em sua totalidade! O loop de "j" passará por todas as suas iterações enquanto "i" for 0.

Para i = 0, j = 0 e depois j = 1.

Depois que tudo de "j" foi executado, o "i" passa para a sua próxima iteração – 1 no caso – e, com o "i" valendo 1, "j" será executado em todas as suas iterações novamente!

Para i = 1, j = 0 e depois j = 1.

Quando essa etapa acaba, “i” já completou o seu range (o seu intervalo sequencial) e o programa para, pois não há mais nenhuma iteração a ser feita. Não há mais laço externo exigindo a iteração de laço interno.

A lógica é a mesma quando usamos 3 loops aninhados. Vamos criar uma matriz 3x3 com “i”, “j” e “k”:

```
for i in range(2):
    for j in range(2):
        for k in range(2):
            print(i, j, k)

```

```
0 0 0
0 0 1
0 1 0
0 1 1
1 0 0
1 0 1
1 1 0
1 1 1

Process finished with exit code 0
```

É exatamente o mesmo funcionamento! Única diferença é que agora temos 2 laços internos. Vamos montar uma tabela e explicar.

Iteração	Valor de i	Valor de j	Valor de k
1ª	0	0	0
2ª	0	0	1
3ª	0	1	0
4ª	0	1	1
5ª	1	0	0
6ª	1	0	1
7ª	1	1	0
8ª	1	1	1

Primeiro, a variável “i” inicia sua sequência assumindo o valor 0. Como existe um segundo loop logo abaixo, o Python passa então a trabalhar com o laço de “j”, que também inicia assumindo 0.

Só que “j” não é o loop mais interno. Dentro dele ainda existe o loop de “k”, que é o responsável por percorrer toda sua sequência primeiro.

Assim, enquanto i = 0 e j = 0, o loop de k realiza todas as suas iterações:

(i=0, j=0, k=0)

(i=0, j=0, k=1)

Somente após k terminar suas iterações é que j pode continuar. Então j avança para 1, e novamente k percorre todo o intervalo:

(i=0, j=1, k=0)

(i=0, j=1, k=1)

Agora que “j” terminou sua sequência inteira, o laço retorna para “i”, que avança para 1.

E o processo começa novamente:

Enquanto i = 1 e j = 0:

(i=1, j=0, k=0)

(i=1, j=0, k=1)

Depois, j = 1, e mais uma vez k executa tudo:

(i=1, j=1, k=0)

(i=1, j=1, k=1)

Como regra geral, os loops mais internos sempre são executados primeiro, completando todas as suas iterações antes que os loops mais externos possam avançar. Em outras palavras, o primeiro for da estrutura é sempre o último a terminar.

Para cada valor assumido pelo loop externo, todos os loops internos percorrem completamente suas sequências. Somente depois que o loop mais interno conclui todas as suas iterações é que o Python permite que o próximo nível avance um passo. Esse processo se repete até que todos os níveis tenham sido totalmente percorridos.

3.6 Manipulando os Laços For – break e continue

O Python permite que modifiquemos como as iterações do laço for vão ocorrer. Nós podemos determinar que algumas dessas iterações não sejam executadas ou, ainda, impor condições para que o laço seja interrompido antes do seu fim.

Para isso, usamos as palavras reservadas break e continue. Vamos explicar o funcionamento de cada uma delas.

3.6.1 Comando break

A palavra **break** dentro de um laço for interrompe a execução do laço por completo. Quando o Python vê a palavra break ele entende que é para imediatamente parar todas as iterações e sair do loop, ainda que fosse possível continuar dentro dele.

Vamos comparar dois programas iguais que vão contar de 0 até 5, mas em um desses programas vamos colocar a condição de que se o número a ser contado for 2, haverá o break e o laço será interrompido.

Programa sem o break – funcionamento normal:

```
for i in range(6):
    print(i)

```

```
0
1
2
3
4
5

Process finished with exit code 0
```

Programa com o break quando o número for 2:

```
for i in range(6):
    if i == 2:
        break
    print(i)

```

```
0
1

Process finished with exit code 0
```

Interessante! Assim que o Python leu o break o loop foi totalmente interrompido. A contagem iria normalmente até 5, mas como colocamos a condição de que o 2 deveria ter um ponto de parada, assim que "i" assumiu o valor 2, a condição foi satisfeita e o break foi executado.

O loop foi interrompido!

ATENÇÃO: A posição do break importa! Se a condição com o break viesse depois do comando print(), o número 2 seria exibido no console e só depois o Python ia chegar no break e interromper o loop. Veja:

```
1    for i in range(6):
2        print(i)
3        if i == 2:
4            break
5
```

```
0
1
2

Process finished with exit code 0
```

Percebam que dessa vez apareceu o 2 no console, pois o loop somente foi interrompido quando o programa chegou na linha 4 e leu o break.

3.6.2 Comando continue

A palavra **continue** dentro de um laço for pula a iteração atual, mas continua o loop a partir da próxima iteração. Em outras palavras: quando o Python lê continue ele para tudo o que está fazendo e recomeça o loop a partir da próxima iteração e continua normalmente.

Ele não interrompe o laço. Somente não executa as linhas de código que estão dentro do loop e abaixo do continue. Vamos mostrar com um exemplo. O nosso programa normal vai contar até 5 começando do 0. Já o programa com continue vai simular um computador birrento que detesta o número 2 e se recusa a exibir esse número no console.

Programa sem o continue – funcionamento normal:

```
1    for i in range(6):
2        print(i)
3
```

```
0
1
2
3
4
5

Process finished with exit code 0
```

Programa com o continue quando o número for 2:

```
1    for i in range(6):
2        if i == 2:
3            print("Eu odeio esse número!")
4            continue
5        print(i)
6
```

```
0
1
Eu odeio esse número!
3
4
5

Process finished with exit code 0
```

Quando i assumiu o valor 2 a condição da linha 2 foi satisfeita e foi impressa a mensagem de desgosto do nosso computador. Depois veio o continue e o print da linha 5 não foi executado! O loop foi reiniciado a partir da próxima iteração (i = 3 agora) e daí continuou normalmente até terminar o loop.

ATENÇÃO: A posição do continue importa! Se a condição com o continue estivesse no final, esse comando não teria impacto nenhum no programa e o número 2 seria exibido antes da mensagem de desgosto:

```python
for i in range(6):
    print(i)
    if i == 2:
        print("Eu odeio esse número!")
        continue

```

```
0
1
2
Eu odeio esse número!
3
4
5

Process finished with exit code 0
```

Nesse exemplo o continue não serviu para nada, pois já estava no final da iteração e não havia nenhum outro comando a ser executado após ele.

3.7 For e Else

É possível combinar um laço for com a instrução **else**, a ser executada quando o laço termina. Nós não vamos abordar a fundo essa interação entre esses comandos no momento, sendo apenas suficiente que você saiba que ele existe e tenha apenas noção básica de seu funcionamento.

A estrutura **for-else** é um pouco diferente da **if-else**. Enquanto na if-else os comandos do else somente são executados quando as condições do if não são atendidas, o else da estrutura for-else é executado quando o laço termina com sucesso.

Ou seja: usamos for-else quando queremos que algo seja executado após o laço for e somente se esse laço foi completado em sua totalidade. Isso significa que se tiver uma condição com break dentro do laço for e esse break for executado, os comandos do else **não** serão executados!

Vamos dar um exemplo simples de um programa que busca uma letra em determinada string:

```
1   texto = "python"
2   letra_procurada = "z"
3
4   for letra in texto:
5       if letra == letra_procurada:
6           print("Letra encontrada!")
7           break
8   else:
9       print("Letra não encontrada.")
10
```

Se a letra não for encontrada na palavra a condição não terá sido satisfeita e o Python não vai ler o break. Como a letra “z” não está na palavra “python”, o laço vai ser completado com todas as suas iterações e o comando else será executado:

```
Letra não encontrada.

Process finished with exit code 0
```

O else não seria executado se a letra buscada estivesse na palavra, pois assim que achasse a letra, a condição da linha 5 estaria satisfeita e o Python iria ler o comando break da linha 7, saindo do loop e ignorando o comando else (pois esse comando faz parte do nosso laço for!)

ATENÇÃO: Esse comportamento somente funciona com o break. O comando continue não impede a execução do else na estrutura for-else.

Repetimos: não se preocupe muito no momento com isso. Somente entenda que essa estrutura existe e que isso pode ser útil em algum momento.

CAPÍTULO 4 – Exercícios com For Loop

4.1 Introdução

Vamos programar um pouco para vermos o uso do laço for na prática. Essa estrutura faz parte do dia a dia do programador e é essencial que estejamos familiarizados e confortáveis em utilizá-la.

4.2 Somando Números Digitados

Vamos criar um programa bem simples: a ideia é usar um laço for para pedir vários inputs de números inteiros do usuário. Ao final vamos exibir no console a soma desses números.

Para isso vamos precisar de uma variável que guarde esses números digitados e realize a soma. Quando queremos atualizar o valor de uma variável usamos a seguinte sintaxe:

variável = variável + número

Então vamos supor que tenhamos uma variável chamada “numero” que tenha recebido 5 e queremos adicionar 10 a ela. Vamos fazer desse jeito:

```
1    numero = 5
2    numero = numero + 10
3
4    print(numero)
5
```

Veja que quando exibimos “numero” no console agora ele terá o valor 15 (5 + 10):

```
15

Process finished with exit code 0
```

Usando essa técnica podemos atualizar o valor das variáveis, isso funciona com as quatro operações matemáticas.

ATENÇÃO: Existe uma forma mais comum na programação de fazer isso. Em vez de usarmos a sintaxe variável = variável + numero, podemos usar o sinal da operação matemática seguida do sinal de igual: variável += numero, por exemplo. Observem os exemplos:

```
numero = 100
numero += 10 # vai somar 10 e virar 110
numero -= 10 # vai subtrair 10 e virar 90
numero *= 10 # vai multiplicar por 10 e virar 1000
numero /= 10 # vai dividir por 10 e virar 10

```

Agora que já sabemos como armazenar e atualizar os valores digitados, vamos para a resolução do problema.

Resolução

Precisamos iniciar uma variável para ser o nosso contador. Ela irá receber os valores digitados pelo usuário e realizar a soma. Em seguida precisamos colocar a função input() dentro do laço for para o usuário digitar os valores e atualizar o valor do contador em cada iteração.

```
soma = 0

for _ in range(3):
    num = int(input("Digite um número inteiro: "))
    soma += num
print(soma)

```

Executando e digitando os números 10, -3 e 5:

```
Digite um número inteiro: 10
Digite um número inteiro: -3
Digite um número inteiro: 5
12

Process finished with exit code 0
```

4.3 Maior e Menor Número Digitado

Nesse exercício o usuário irá informar a idade de 5 pessoas diferentes. O nosso programa deve ser capaz de identificar quais foram a maior e a menor idades digitadas e exibir no console.

Resolução

Vamos criar um laço for para receber 5 inputs de números inteiros (idades). Temos de ter uma variável para armazenar a maior idade e outra para armazenar a menor. Conforme as iterações vão acontecendo e novas idades são digitadas, as variáveis devem comparar o seu valor atual com o novo número digitado para saber se serão atualizadas (se o valor digitado foi maior que a variável maior ou menor que a variável menor).

```python
maior = 0
menor = 0

for i in range(1, 6):
    idade = int(input(f"Idade da {i}ª pessoa: "))
    if i == 1:
        maior = idade
        menor = idade
    else:
        if idade > maior:
            maior = idade
        if idade < menor:
            menor = idade

print(f"A menor idade digitada foi {menor}.")
print(f"A maior idade digitada foi {maior}.")
```

Vamos explicar bem: no início criamos as variáveis para a maior e a menor idade digitada. Ambas começam com 0.

No loop for, vamos começar a pegar as idades digitadas pelo usuário. Na primeira iteração, a primeira idade digitada será considerada tanto a maior quanto a menor idade (afinal somente temos uma idade até agora).

O nosso range foi de 1 a 6, ou seja, a variável “i” irá receber os valores 1, 2, 3, 4, 5 em cada iteração. Na primeira iteração, portanto, “i” vale 1! Então quando “i” for 1, tanto “maior” quanto “menor” receberão o primeiro valor digitado pelo usuário.

Nas demais iterações, se a idade digitada for maior que aquela armazenada na variável “maior”, a variável é atualizada com o novo valor. A mesma lógica para a menor: se a idade digitada for menor do que o valor da variável “menor”, há a substituição

Executando o programa e digitando os valores 5, 27, 90, 13 e 2 temos o seguinte resultado:

```
Idade da 1ª pessoa: 5
Idade da 2ª pessoa: 27
Idade da 3ª pessoa: 90
Idade da 4ª pessoa: 13
Idade da 5ª pessoa: 2
A menor idade digitada foi 2.
A maior idade digitada foi 90.

Process finished with exit code 0
```

CAPÍTULO 5 – Estruturas de Repetição: Laço While

5.1 Definição e Sintaxe

O while loop – laço while – permite a repetição do código até que determinada condição de parada seja atingida.

A ideia por trás dessa estrutura de repetição é a mesma do laço for, mas enquanto o laço for passa a instrução para o programa executar determinado bloco de código um número preestabelecido de vezes, o laço while manda repetir o bloco até que uma condição seja satisfeita.

A sintaxe do while loop é a seguinte:

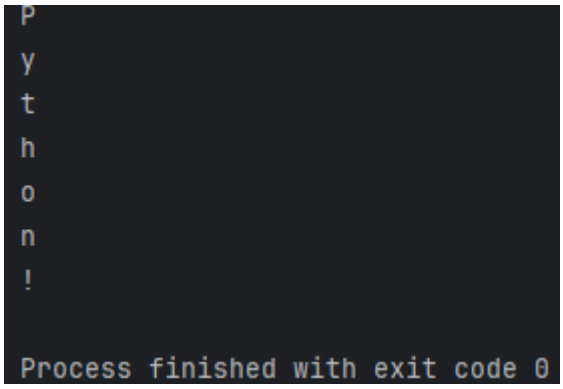
while condição:

instruções

Vamos dar um exemplo usando o while para iterar sobre os caracteres de uma string:

```
1  texto = "Python!"
2  indice = 0
3
4  while indice < len(texto):
5      print(texto[indice])
6      indice += 1
7
```

Resultado:



Vamos explicar com muita calma e atenção o que foi feito. A estratégia para acessar os caracteres da string “texto” foi usar a funcionalidade de acesso aos índices da string - nomedastring[índice] – conforme estudamos aprofundadamente no Capítulo 2.

Para isso funcionar, precisamos de 2 pré-requisitos:

a) Uma variável que receba os números dos índices – variável “índice” da linha 2;
b) Uma forma de incrementar essa variável a cada iteração do laço. Como desejamos acessar os caracteres um a um, precisamos fazer com que, a cada repetição do while, o índice avance para a próxima posição da string. Isso é feito por meio da incrementação na linha 6.

Então sabemos que precisamos ir do 0 até o último índice da string. Aprendemos no Capítulo 2 que a função len() nos retorna um número que indica o tamanho da string, ou seja, o seu número de índices.

Essa função é perfeita para criarmos a nossa condição. Veja a linha 4. Demos a seguinte instrução para o programa:

“Enquanto a variável “indice” for menor que o tamanho da string, mostre no console o índice atual (começa com 0, pois estabelecemos isso na linha 2) e depois passe para o próximo (linha 6 quando colocamos o +=1).

Quando “indice” for igual ao tamanho da string, a nossa condição de parada é atingida (a condição para executar o laço era “indice” ser menor que o tamanho da string) e o programa finaliza.”

No caso, a string tem 7 caracteres, então len(texto) irá retornar 7 como valor.

Veja a tabela a seguir para entender o que está acontecendo em cada iteração:

Iteração	Valor de indice	Condição	Caractere
1ª	0	0 < 7 → **True**	P
2ª	1	1 < 7 → **True**	y
3ª	2	2 < 7 → **True**	t
4ª	3	3 < 7 → **True**	h
5ª	4	4 < 7 → **True**	o
6ª	5	5 < 7 → **True**	n
7ª	6	6 < 7 → **True**	!
-	7	7 < 7 → **False**	Laço encerrado

5.2 Diferenças Entre For e While e Quando Usar

Tanto o for loop quanto o while loop têm a mesma função – criar laços de repetição usados para repetir trechos de código e percorrer iteráveis.

Quando sabemos exatamente o número de repetições que desejamos, o programador geralmente dá preferência para o laço for. Vimos essa situação no exercício “Maior e Menor Número Digitado” do Capítulo 4.

Naquele exercício sabíamos que o usuário tinha de digitar 5 números relativos às idades de 5 pessoas diferentes. Como sabemos que serão exatamente 5 repetições, é mais adequado usar o laço for para isso (mesmo assim não é impossível usar while nesse caso!).

Por outro lado, quando não sabemos exatamente quantas vezes o código deve ser repetido, o uso do while passa a ser mais interessante, pois o código continuará rodando enquanto a condição estabelecida não for atingida, independente do número de iterações realizadas.

Em suma:

a) usamos **for** quando sabemos quantas repetições serão necessárias;
b) usamos **while** quando a repetição depende de uma condição que pode variar.

Cabe ao programador escolher qual ferramenta é mais conveniente para o seu código.

Outra diferença relevante está na própria forma como o laço é desenvolvido. Quando usamos o **for** para iterar sobre um iterável, por exemplo, uma string ou a função range, o Python automaticamente vai passando para o próximo item. A incrementação é automática.

Já o **while** não funciona assim. O programador é que deve iniciar uma variável para armazenar em qual momento da iteração o programa se encontra e determinar uma lógica que faça o programa seguir para o próximo item (como bem vimos no exemplo desse Capítulo).

Assim, para o while, o programador deve:

a) inicializar manualmente a variável de controle;
b) definir a condição de parada;
c) garantir que essa variável seja atualizada corretamente a cada iteração.

5.3 Break e Continue

Os comandos **break** e **continue** também podem ser utilizados dentro de um laço while. Funcionam exatamente como no laço for, por isso não vamos ficar repetindo e vamos focar apenas em relembrar.

Quando o Python lê **continue** ele entende que essa iteração tem de ser pulada. Tudo o que estiver abaixo do continue não será executado e o laço irá reiniciar, passando para a iteração seguinte.

Já o comando **break** quebra o loop. Quando o Python lê break ele interrompe o laço e sai dele. As iterações chegam ao seu fim.

ATENÇÃO: O programador deve lembrar que no laço while ele é que tem de atualizar a variável de controle para que o Python realize corretamente as próximas iterações. Se essa variável não for atualizada podemos chegar numa situação de loop infinito. Então, lembre-se que a atualização da variável sempre tem de vir antes da palavra continue, senão ela não será executada!

5.4 While True – Loop Infinito

Chamamos de while True a situação em que um laço while possui uma condição que jamais será atingida, de forma que o laço irá ser executado eternamente até o programa ser fechado. É o loop infinito!

O loop infinito é chamado de condição while True porque se escrevermos um laço while com essa condição ele nunca terminará. Veja os exemplos:

```
controle = 0

while controle < 5:
    print("Ainda não terminei!")

```

Aqui temos um loop infinito por erro de lógica. Não estamos atualizando o valor de “controle”. A cada iteração controle permanecerá 0. Como 0 sempre será menor que 5, temos um loop infinito. 0 < 5 sempre será verdade (while True). Veja o resultado no console:

```
Ainda não terminei!
Ainda não terminei!
Ainda não terminei!
Ainda não terminei!
Ainda não terminei!
Ainda não terminei!
Ainda não terminei!
Ainda não terminei!
Traceback (most recent call last):
  File "C:\Users\rafae\PycharmProjects\Tutorial\.venv\while_loop.py", line 4, in <module>
    print("Ainda não terminei!")
    ~~~~~^^^^^^^^^^^^^^^^^^^^^^^
KeyboardInterrupt

Process finished with exit code -1073741510 (0xC000013A: interrupted by Ctrl+C)
```

O comando print() será executado para sempre. O programa somente parou porque foi interrompido manualmente pelo usuário, caso contrário ele permaneceria sendo executado indefinidamente.

É o que acontece quando usamos while True:

```
while True:
    print("Sou um loop Infinito!")

```

Aqui a mensagem “Sou um loop Infinito!” também será exibida incessantemente, como aconteceu no exemplo anterior.

Então, podemos chegar a um loop infinito de duas formas:

a) usando explicitamente a condição while True;
b) o programa contém erro de lógica que irá impedir que a condição de parada seja atingida.

Lembrando que, na prática, ambas as hipóteses produzem o mesmo resultado.

5.4.1 Problema do Loop Infinito e Como Evitar

Conforme explicamos, o loop infinito pode ser uma situação indesejável, pois o programa ficará preso naquele laço while e não fará mais nada. Nenhuma outra linha de código fora do laço while será lida, pois o Python nunca vai conseguir chegar lá.

Para evitar que ocorra um loop infinito indesejado, o programador deve se atentar para a lógica da condição de parada e para a atualização de sua variável de controle. É necessário que a condição de parada tenha condições de ser satisfeita.

Estamos sendo repetitivos porque o erro lógico é algo muito comum de acontecer, especialmente na atualização incorreta da variável de controle. Se não tomar cuidado, o programador pode criar um loop infinito que irá impedir o correto funcionamento de seu código.

5.4.2 Quando Usar Loop Infinito

O uso do loop infinito (while True) pode ser útil em alguns casos. Nem sempre será um problema, podendo nos ajudar no desenvolvimento do nosso programa.

Para que isso funcione, vamos precisar programar uma condição de parada usando o comando break. Já que o laço será infinito segundo a lógica de programação, precisamos estipular uma condição que, caso aconteça, apresente o comando break para o Python poder interromper o laço.

Um bom exemplo de uso correto do while True é quando queremos criar um menu com opções para o usuário. Como o laço é infinito, enquanto o usuário desejar usar o programa o menu ficará aberto e recebendo instruções.

Para codificar corretamente esse loop, o menu deve conter alguma instrução para o encerramento do programa, por exemplo, digitar 0 para fechar o programa. Assim, o usuário poderá utilizar as opções do menu indefinidamente, até que decida digitar 0 e fechá-lo.

Vamos criar um menu bem simples que tenha as seguintes funcionalidades:

a) digitar 1 para dizer oi;
b) digitar 2 para exibir uma mensagem;
c) digitar 0 para sair do programa.

Escrevendo esse código:

```
while True:
    print("MENU DE OPÇÕES")
    print("1 - Dizer olá")
    print("2 - Mostrar uma mensagem")
    print("0 - Sair do programa")

    opcao = input("Escolha uma opção: ")

    if opcao == "1":
        print("Olá! Seja bem-vindo ao programa.")

    elif opcao == "2":
        print("Este é um exemplo de menu usando while True.")

    elif opcao == "0":
        print("Encerrando o programa...")
        break

    else:
        print("Opção inválida. Tente novamente.")

```

Vamos analisar com atenção o que está sendo feito para entendermos bem o funcionamento do loop infinito.

Ao executar o programa, será exibido o menu com as mensagens que estão nas linhas 2, 3, 4 e 5, além do input da linha 7:

```
MENU DE OPÇÕES
1 - Dizer olá
2 - Mostrar uma mensagem
0 - Sair do programa
Escolha uma opção:
```

O usuário poderá digitar a sua opção. Nas linhas de código seguinte temos estruturas condicionais (if -elif – else) que vão executar comandos conforme a escolha do usuário.

Depois que uma opção for escolhida, o print() correspondente será executado e exibido no console e, por ser um laço infinito, o programa vai retornar ao menu de opções, exibindo todas as opções novamente, até que o usuário interrompa o programa ou digite 0 para fechar.

Vamos ver os exemplos de cada situação. Primeiro exemplo – usuário digitou 1:

```
MENU DE OPÇÕES
1 - Dizer olá
2 - Mostrar uma mensagem
0 - Sair do programa
Escolha uma opção: 1
Olá! Seja bem-vindo ao programa.
MENU DE OPÇÕES
1 - Dizer olá
2 - Mostrar uma mensagem
0 - Sair do programa
Escolha uma opção:
```

Foi exibida a mensagem de dizer olá (print() da linha 10). Imediatamente depois o programa já voltou para o menu e aguarda a nova escolha.

Segundo exemplo – usuário digitou 2:

```
MENU DE OPÇÕES
1 - Dizer olá
2 - Mostrar uma mensagem
0 - Sair do programa
Escolha uma opção: 2
Este é um exemplo de menu usando while True.
MENU DE OPÇÕES
1 - Dizer olá
2 - Mostrar uma mensagem
0 - Sair do programa
Escolha uma opção:
```

Agora a mensagem a ser exibida foi aquela do print() da linha 13. Assim que ela foi exibida, o programa voltou para o início do laço while e agora podemos escolher uma nova opção.

Terceiro exemplo – usuário digitou uma opção inválida:

```
Escolha uma opção: Dizer olá
Opção inválida. Tente novamente.
MENU DE OPÇÕES
1 - Dizer olá
2 - Mostrar uma mensagem
0 - Sair do programa
Escolha uma opção:
```

Nesse caso, o usuário desobedeceu às instruções do programa e, em vez de digitar 1 para exibir a mensagem de olá, resolveu escrever por extenso o que desejava. Esse erro fez com que o else da linha 19 fosse executado, com a exibição de mensagem de opção inválida. Logo após a mensagem, o menu foi exibido novamente, por se tratar de while True (loop infinito).

Como podemos bem ver, é possível passar o dia todo executando esses comandos. O laço não terminará enquanto não for encontrado um break. No caso, o break se encontra na opção 0 (linha 17). Vamos ao quarto exemplo – usuário digitou 0:

```
MENU DE OPÇÕES
1 - Dizer olá
2 - Mostrar uma mensagem
0 - Sair do programa
Escolha uma opção: 0
Encerrando o programa...

Process finished with exit code 0
```

Foi exibida a mensagem de encerramento (print() da linha 16) e logo após o Python chegou no break. O programa foi encerrado e o loop não mais será executado.

5.5 Laço While Aninhado – Nested While Loop

Assim como acontece com o laço for, também é possível criar estruturas com laços while dentro de outros laços while. Essa técnica é chamada de laço while aninhado.

O conceito é exatamente o mesmo que já estudamos anteriormente com o for aninhado: temos um laço externo, responsável pelo funcionamento geral do programa, e um laço interno, que executa uma tarefa específica dentro de cada iteração do laço externo.

Assim como ocorre no laço for, o while mais interno vai ser executado em sua totalidade antes da execução do while externo.

Vamos ver o funcionamento dessa estrutura em um programa que simula o controle de tentativas de login. Para simplificar, a senha estará codificada em nosso programa, mas lembre-se que isso não pode acontecer em situações do mundo real, pois tornaria a senha totalmente insegura.

Nosso programa irá funcionar assim: tem uma senha cadastrada para o usuário e ele deverá digitá-la. Se a senha estiver correta ele pode acessar o sistema, caso contrário ele tem até 3 tentativas para acertar a senha. Se nessas 3 tentativas a senha estiver incorreta, o programa irá bloquear o acesso por segurança.

```
1   password = "python123"
2   tentativas = 0
3
4   while tentativas < 3:
5       senha = input("Digite a senha: ")
6
7       while senha != password:
8           tentativas += 1
9           print("Senha incorreta.")
10
11          if tentativas == 3:
12              print("Acesso bloqueado.")
13              break
14
15          senha = input("Digite a senha novamente: ")
16
17      if senha == password:
18          print("Acesso liberado!")
19          break
20
```

Nesse exemplo temos a senha do usuário na variável “password”. O programa também irá contar o número de tentativas de escrever a senha.

O laço mais externo (linha 4) irá dar início ao programa pedindo a senha. Caso a senha digitada esteja correta, o acesso será liberado. Se a senha for digitada corretamente já na primeira tentativa o laço interno nem será executado!

Entretanto, se o usuário errar a senha, o laço interno é iniciado. Enquanto a senha for digitada de forma incorreta, o laço interno irá contar as tentativas. Caso haja 3 tentativas frustradas, a mensagem de bloqueio será exibida e o laço interno será finalizado.

Nessa hipótese a variável "tentativas" terá 3 como valor, o que também vai finalizar a execução do laço externo, pois a condição dele é: tentativas < 3 e teremos 3 < 3 (o que é falso)!

Vamos executar o programa e analisar caso a caso! Caso 1 – usuário acertou a senha:

```
Digite a senha: python123
Acesso liberado!

Process finished with exit code 0
```

A senha está correta e o acesso foi liberado. Não foi utilizado o laço interno.

Caso 2 – usuário erra primeiro, mas depois acerta antes das 3 tentativas:

```
Digite a senha: 1
Senha incorreta.
Digite a senha novamente: python123
Acesso liberado!

Process finished with exit code 0
```

A primeira senha digitada foi "1", que está incorreta. O laço interno foi acionado com a mensagem para digitar a senha novamente e contagem da tentativa incorreta.

Caso 3 – usuário erra a senha 3 vezes:

```
Digite a senha: 1
Senha incorreta.
Digite a senha novamente: 2
Senha incorreta.
Digite a senha novamente: 3
Senha incorreta.
Acesso bloqueado.

Process finished with exit code 0
```

O número de tentativas chegou ao seu fim. O acesso foi bloqueado e os laços não mais serão executados.

5.6 While e Else

Assim como estudamos no laço for, também é possível usar else junto com o while. O funcionamento é exatamente o mesmo: o bloco de código do else somente será executado caso o laço while termine normalmente e sem interrupções, ou seja, sem o uso do comando break.

CAPÍTULO 6 – Exercícios com While Loop

6.1 Validação de Entrada – Básico

Vamos criar um programa simples usando laço while que seja capaz de validar o input do usuário. A ideia é que o usuário digite um número considerado válido. O programa irá considerar como válido as seguintes condições:

a) deve ser digitado um número inteiro;
b) esse número deve ser maior que 0

Enquanto o usuário não digitar um número válido o programa deve continuar rodando e pedindo que seja digitado o número. Quando o usuário digitar um número válido, o programa deverá exibir a mensagem: "Número válido registrado!" e encerrar a sua execução.

Resolução

Para resolver o problema precisamos de um número inteiro que seja maior do que 0. Não pode ser negativo e nem igual a 0.

Enquanto o número digitado for menor ou igual a 0, o programa deve continuar pedindo nova entrada. Para isso usamos um laço while com essa condição.

Escrevendo o código:

```
entrada = int(input("Digite um número inteiro maior que 0: "))

while entrada <= 0:
    entrada = int(input("Valor inválido! Digite um número inteiro maior que 0: "))

print("Número válido registrado!")

```

Executando e testando com os números -3, 0 e 5:

```
Digite um número inteiro maior que 0: -3
Valor inválido! Digite um número inteiro maior que 0: 0
Valor inválido! Digite um número inteiro maior que 0: 5
Número válido registrado!

Process finished with exit code 0
```

ATENÇÃO: É muito importante que o input da linha 4 (dentro do loop while) seja convertido para int, para que a condição entrada <= 0 possa ser analisada. Se não houver a conversão, o input será **str** por padrão e é impossível verificar se uma string é menor ou igual a 0. O programa iria quebrar!

6.2 Validação de Entrada - Aprimorando

Vamos melhorar o programa do item 6.1! No programa anterior, caso o usuário digitasse algo que não fosse um número inteiro o programa quebraria e pararia de funcionar. A ideia é melhorar essa validação para que o programa possa receber valores diferentes de números inteiros e continuar funcionando.

Para isso, não podemos já converter a entrada do usuário para int diretamente sem antes termos a certeza de que isso pode ser feito sem quebrar o programa.

Resolução

Para resolver esse problema vamos precisar dos métodos de string que estudamos no Capítulo 1. Como todo input do usuário vem como string, precisamos ter certeza de que o usuário digitou apenas números.

O método de string que faz isso é o isdigit() – sintaxe nomedastring.isdigit() – que nos retornará True se a string for composta apenas por números. Com esse método vamos garantir que a entrada do usuário foi somente com números inteiros e positivos, pois as

frações possuem ponto (e ponto não é um número) e números negativos possuem sinal de negativo (o que também não é um número).

Então com o isdigit() já conseguimos garantir que a validação somente vai aceitar números inteiros e positivos!

Então o input do usuário tem de atender a 2 condições **simultâneas**:

a) isdigit() == True;
b) entrada > 0

Então nossa lógica para o laço while é a seguinte: **enquanto a entrada NÃO for apenas números OU se ela for menor ou igual a 0**, o programa continua pedindo nova entrada.

Vamos escrever o código:

```
1  entrada = input("Digite um número inteiro maior que 0: ")
2
3  while not entrada.isdigit() or int(entrada) <= 0:
4      entrada = input("Valor inválido! Digite um número inteiro maior que 0: ")
5
6  print("Número válido registrado!")
7
```

Vamos testar usando 0 (inválido), -1 (inválido) e 1 (válido):

```
Digite um número inteiro maior que 0: 0
Valor inválido! Digite um número inteiro maior que 0: -1
Valor inválido! Digite um número inteiro maior que 0: 1
Número válido registrado!

Process finished with exit code 0
```

ATENÇÃO: Nesse caso específico a condição da linha 3 int(entrada) <= 0 poderia ser escrita somente como int(entrada) == 0, pois o isdigit() já irá garantir que o número será positivo, já que o sinal de menos (-) dos números negativos já

invalida o isdigit() (afinal, o sinal de menos não é um número então a string não é composta apenas por números).

6.3 Caixa Eletrônico

Nesse exercício você deve criar um programa que simule o funcionamento de um caixa eletrônico. Uma vez executado o programa, deve ser exibido um menu ao usuário com as seguintes opções:

1 - Consultar saldo

2 – Sacar dinheiro

3 – Depositar dinheiro

4 – Sair

Enquanto o usuário não decidir sair o menu deve continuar funcionando. Caso o usuário escolha as opções sacar ou depositar, o programa deve perguntar o quanto ele deseja retirar ou colocar na conta. Isso deve atualizar o saldo no programa.

Vamos considerar como saldo inicial o valor de R$1.000,00. O banco não vai aceitar saldo negativo! Não pode sacar mais do que estiver disponível na conta!

Para facilitar o desenvolvimento do programa, vamos considerar que o usuário somente irá digitar valores int e float válidos corretos. Mas nada impede que você crie um programa com validação de dados robusta para treinar tudo o que já estudamos.

Resolução

Para resolver o problema temos de começar com uma variável para o saldo, com valor inicial de R$1.000,00.

Em seguida, vamos usar um laço while True para fazer o menu. Cada opção do menu terá funcionalidade própria que será executada com base no que o usuário escolher.

Vamos pegar o input do usuário e usar estruturas de condição para executar as funcionalidades que vamos programar. Lembrando que as escolhas do usuário deverão afetar o saldo da conta.

Escrevendo o nosso código:

```
saldo = 1000.00

while True:
    print("Por favor, informe a operação desejada.")
    print("1 - Consultar saldo.")
    print("2 - Sacar dinheiro.")
    print("3 - Depositar dinheiro.")
    print("4 - Sair.")
    escolha = int(input("Opção: "))

    if escolha == 1:
        print("Saldo: R$", saldo)

    elif escolha == 2:
        print("Por favor, informe a quantidade a ser retirada.")
        valor = float(input("Valor:R$ "))
        if valor > saldo:
            print("Saldo insuficiente.")
        else:
            print("Valor retirado com sucesso.")
            saldo -= valor

    elif escolha == 3:
        print("Por favor, informe a quantidade a ser depositada.")
        valor = float(input("Valor:R$ "))
        print("Valor depositado com sucesso.")
        saldo += valor

    elif escolha == 4:
        print("Obrigado. Volte sempre.")
        break

```

É bem simples: temos um menu que vai funcionar até o usuário digitar 4. A opção 4 leva ao break e o programa encerra.

A opção 1 somente exibe o saldo atual.

A opção 3 pergunta a quantidade a ser depositada e adiciona ao saldo, aumentando-o.

A opção 4 pergunta o valor a ser retirado. Como a conta não pode ficar negativa, esse valor não pode ser maior que o saldo atual. Se o saldo for suficiente, o valor é descontado do saldo, reduzindo-o.

6.4 Considerações Finais

Com isso, sabemos como lidar com laços de repetição. Na próxima parte do tutorial vamos estudar as estruturas de dados do Python e os laços **for** e **while** serão muito úteis para trabalhar com elas.